

DORU ANASTASIU POPESCU

BAZELE PROGRAMĂRII

JAVA DUPĂ C++



2019 - Editura L&S Soft, București

Copyright 2019 © L&S SOFT

Toate drepturile asupra acestei lucrări aparțin editurii L&S SOFT. Reproducerea integrală sau parțială a textului din această carte este posibilă doar cu acordul în scris al editurii L&S SOFT.

ISBN: 978-973-88037-9-4

ATENȚIE!

După confirmarea plății, fiecare carte poate fi descărcată de maximum 5 ori și este disponibilă 30 de zile.

Fiecare PDF este securizat în 28 de zone cu watermark invizibil (id comandă, e-mail, nume) pentru a nu putea fi distribuit pe alte căi virtuale.

Editura L&S SOFT

S.C. L&S Soft Management Impex S.R.L.

Adresa: Aleea Aviației nr. 10, Voluntari, Ilfov

Mobil: 0727.731.947

E-mail: comenzi@ls-infomat.ro

www.ls-infomat.ro

www.manuale-de-informatica.ro

Biblioteca Digitală de Informatică "Tudor Sorin"

www.infobits.ro

Cărți în format electronic

<https://ebooks.infobits.ro>



CUPRINS

INTRODUCERE	5
1. NOȚIUNEA DE CLASĂ	7
2. PROGRAME JAVA CU INSTRUCȚIUNI REPETITIVE	14
3. TABLOURI	19
4. PROGRAME JAVA CU O SINGURĂ CLASĂ. DATE MEMBRU ȘI METODE	29
5. PRINCIPIILE PROGRAMĂRII ORIENTATE PE OBIECTE	43
6. CLASA MATH, CLASA STRING ȘI CLASE ÎNFĂȘURĂTOARE	57
7. MOȘTENIREA CLASELOR	81
8. CLASE ABSTRACTE ȘI INTERFEȚE	99
9. EXCEPȚII. CLASA STRINGTOKENIZER. FIȘIERE TEXT	109
10. NOȚIUNI INTRODUCTIVE DE PROGRAMARE VIZUALĂ	121
11. COMPONENTE FOLOSITE ÎN PROGRAMAREA VIZUALĂ	130
12. FIRE DE EXECUȚIE	138
13. APPLET-URI CU FIRE DE EXECUȚIE	155
14. BIBLIOGRAFIE	167



Doru Anastasiu Popescu a absolvit Colegiul Național "Ion Minulescu", profilul Matematică - Fizică în anul 1986, apoi Universitatea din București, Facultatea de Matematică - secția matematică în 1992 și informatică în 2000. Titlul de doctor în informatică l-a obținut în anul 2012 la Universitatea din Pitești, Facultatea de Matematică-Informatică.



Activitatea didactică a desfășurat-o la: Colegiul Național "Radu Greceanu" din Slatina începând cu anul 1992, Universitatea din Pitești din 2003 și USAMV – Slatina din 2012. În 2016 a obținut titlul de conferențiar universitar la Universitatea din Pitești, Departamentul de Matematică - Informatică.

Domnul profesor **Doru Anastasiu Popescu** a publicat peste 25 de manuale și culegeri de Informatică și TIC pentru gimnaziu și liceu, peste 60 de articole științifice din domeniile: combinatorică, teoria grafurilor, algoritmi, limbaje de programare, verificare și testare, programare web, aplicații web, e-learning, programarea roboților.

Începând cu anul 1996 activitatea didactică a domnului profesor este completată cu participarea în Comisia Centrală a Olimpiadei Naționale de Informatică și colectivul de pregătire a Lotului Național de Informatică, unde a propus numeroase probleme de informatică, pentru gimnaziu și liceu. Multe din aceste probleme se găsesc pe site-urile de pregătire a elevilor pentru olimpiade și concursuri (infoarena, .campion, etc.). Între 2000 și 2014 a participat la peste 10 concursuri internaționale ca team leader sau deputy leader al echipei României (CEOI 2001- Ungaria, CEOI 2002- Slovacia, CEOI 2003- Germania, CEOI 2004- Polonia, CEOI 2005- Ungaria, BOI 2006- Cipru, BOI 2007- Moldova, CEOI 2011- Polonia, BOI 2012- Serbia, Turneul Shumen 2014 – Bulgaria). În 2016 și 2018 a participat ca profesor coordonator al unei echipe din Romania la Olimpiada Mondială de Robotică din India, respectiv Thailanda, iar în 2019 la un Open din First Lego League în Uruguay. Concursuri renumite, precum INFO-OLTENIA, "Grigore Moisil", INFO-OLT, greceanu.ro l-au avut ani la rând ca președinte sau coordonator pe domnul profesor Doru Anastasiu Popescu.

DESPRE "JAVA DUPĂ C++"

Această carte își propune să ajute cititorul să învețe elementele de bază ale limbajului JAVA și POO (Programare Orientată pe Obiecte) pornind de la cunoștințe elementare de C++.

Lucrarea se adresează elevilor din clasa a XI-a de la profilul Matematică-Informatică, intensiv informatică, studenților de la facultățile care au în programă cursul de POO sau cursuri ce conțin elemente de POO și Programare Paralelă, profesorilor de informatică, precum și dezvoltatorilor de aplicații ce folosesc acest tip de programare.

Capitolele au în general aceeași structură:

- noțiuni de teorie, sumar prezentate și bazate pe elemente din C++;
- exemple și probleme rezolvate;
- probleme propuse ce se rezolvă folosind noțiunile prezentate la începutul capitolului și ideile din problemele rezolvate.

Trecerea de la programarea în C++ bazată pe tipuri de date și funcții la clase și obiecte este realizată pas cu pas, pentru a oferi cititorului noțiunile și modul de aplicare a lor într-o variantă cât mai naturală. Pentru familiarizarea cu noțiunile prezentate în această carte recomand analizarea și verificarea programelor rezolvate, după care rezolvarea problemelor propuse.

Pentru verificarea programelor s-a ales modul de lucru cu linia de comandă (cmd.exe) și mediul de programare NetBeans.

Mulțumiri doamnei profesoare Rodica Rădulescu pentru observațiile și discuțiile constructive legate de această carte.

Doru Anastasiu Popescu

1. Noțiunea de clasă

Noțiuni introductive

Un program în limbajul Java este format din una sau mai multe clase.

Clasa conține: - date membru (exemplu: `int a, b`)

- metode (funcții, exemplu: `int f(int a, char b) {...}`)

Clasele conduc la obținerea unor obiecte (variabile) – clasa poate fi privită ca un tip de date complex, iar datele și metodele unui obiect `o` sunt folosite în modul următor: `o.a, o.b, o.f(x,y)`;

Într-un program Java pot exista mai multe clase (`c1, c2, ...`), însă doar una dintre acestea se lansează în executare prima dată (cea care conține metoda `main` și dă numele fișierului ce conține programul, numită clasă principală). Nu are importanță ordinea în care sunt scrise clasele în program.

Pentru un program ce conține clasele `c1, c2, c3`, cu `c1` clasă principală, numele fișierului ce conține programul va fi `c1.java`.

După compilarea programului se creează automat fișierele `c1.class, c2.class, c3.class`. Lansarea în executare a programului presupune utilizarea fișierului `c1.class`.

Operații specifice cu un program salvat cu numele `c1.java`, de la linia de comandă `cmd.exe`:

Compilare => `javac cale/c1.java`

Executare => `java -classpath cale c1`

Observații

1. Echivalentul bibliotecilor din C++, în Java, sunt pachetele. Acestea conțin clase ce pot fi utilizate fără a le defini, dacă se folosește comanda: **import nume_pachet;** (exemplu: **import java.util.Scanner;**), la începutul programului.
2. Majoritatea instrucțiunilor și tipurilor de date elementare din C++ se folosesc și în Java.
3. Ordinea de scriere a metodelor și datelor nu are importanță.

Exemplu

```
class exemplu{  
  
    public static void main(String[] args){  
  
        System.out.println("exemplu java");  
  
    }  
  
}
```

Programul anterior conține o singură clasă (evident cea principală), numele fișierului ce-l conține trebuie să fie *exemplu.java*. Efectul acestui acestui program este afișarea textului `exemplu java` în consola input/output.

Tipuri de date simple

Tipurile de date simple din C++ apar și în Java, diferit este tipul **boolean** care are ca corespondent în C++ tipul **bool**.

1. Tipuri întregi:

- **byte**
- **short**
- **int**
- **long**

2. Tipuri reale:

- **float**
- **double**

3. Tipul caracter:

- **char**

4. Tipul **boolean**:

- cu doua valori: - **true**
 - **false**

Exemplu:

```
boolean x=true;
```

Observații

1. Operatorii sunt la fel ca în C++.
2. Instrucțiunile liniare, alternative și repetitive din C++ sunt cu aceeași sintaxă și în Java.
3. Mulțimile de valori pentru aceste tipuri de date și operatorii sunt la fel ca în C++.

Citirea datelor

Citirea datelor în Java poate fi realizată folosind clasa **Scanner** definită în pachetul **java.util.Scanner**. Etapele ce trebuie parcurse pentru citirea unor date sunt următoarele:

1) Importarea unui pachet:

Linia

```
import java.util.Scanner;
```

va fi scrisă la începutul programului.

2) Definirea unui obiect:

```
Scanner cin=new Scanner(System.in);
```

înainte de citirea datelor.

3) Folosirea metodelor de citire a datelor:

```
cin.nextInt();                    =>        returnează un număr de tip int citit de la  
consolă
```

`cin.nextDouble();` => returnează un număr de tip **double** citit de la consolă

`cin.nextFloat();` => returnează un număr de tip **float** citit de la consolă

`cin.nextLine();` => returnează un șir de caractere citit de la consolă

Probleme rezolvate cu instrucțiuni liniare și alternative

1. Se dă n număr natural cu cel mult 3 cifre. Afișați pătratul și cubul lui n.

Exemplu

Intrare	Ieșire
n=10	patratul lui 10 este 100 cubul lui 10 este 1000

Soluție

```
import java.util.Scanner;

public class P1 {

    public static void main(String[] args) {

        int n;

        Scanner cin=new Scanner(System.in);

        System.out.print("n=");

        n=cin.nextInt();

        System.out.println("patratul lui "+n+" este "+n*n);

        System.out.println("cubul lui "+n+" este "+n*n*n);

    }

}
```

2. Se dau două numere întregi a și b cu maxim 9 cifre fiecare. Afișați a și b în ordine crescătoare.

Exemplu

Intrare	Ieșire
a=10 b=8	8 10

Soluție

```
import java.util.Scanner;

public class P2 {

    public static void main(String[] args) {

        int a,b;

        Scanner cin=new Scanner(System.in);

        System.out.print("a=");

        a=cin.nextInt();

        System.out.print("b=");

        b=cin.nextInt();

        if(a>b)

            System.out.println(b+" "+a);

        else

            System.out.println(a+" "+b);

    }

}
```

3. Se dau trei numere naturale a, b, c cu maxim 4 cifre fiecare. Verificați dacă a, b, c sunt lungimile laturilor unui triunghi dreptunghic.

Exemplu

Intrare	Ieșire
a=10 b=8 c=6	DA

Soluție

```
import java.util.Scanner;

public class P4 {

    public static void main(String[] args) {

        int a,b,c;

        Scanner cin=new Scanner(System.in);

        System.out.print("a=");

        a=cin.nextInt();

        System.out.print("b=");

        b=cin.nextInt();

        System.out.print("c=");

        c=cin.nextInt();

        if(a*a==b*b+c*c || b*b==c*c+a*a || c*c==a*a+b*b)

            System.out.println("DA");

        else

            System.out.println("NU");

    }

}
```

Probleme propuse cu instrucțiuni liniare și alternative

1. Se dă n număr natural cu exact 3 cifre. Afișați suma cifrelor lui n .

Exemplu

Intrare $n=126$	Ieșire 9
---------------------------	--------------------

2. Se dau a și b numere naturale cu maxim 9 cifre fiecare . Verificați dacă b este divizor pentru a.

Exemplu

Intrare	Ieșire
a=120 b=10	DA

3. Se dau a, b, c numere întregi cu cel mult 17 cifre fiecare. Verificați dacă numerele a, b, c sunt consecutive – crescător.

Exemplu

Intrare	Ieșire
a=126 b=127 c=128	9

4. Se dau a, b, c, d numere întregi cu cel mult 17 cifre fiecare. Verificați dacă numerele a, b, c, d formează o progresie aritmetică în această ordine.

Exemplu

Intrare	Ieșire
a=126 b=129 c=132 d=135	DA

5. Se dă n număr natural cu exact 3 cifre. Afișați cel mai mare număr ce se poate forma cu două din cifrele lui n.

Exemplu

Intrare	Ieșire
n=326	63

2. Programe Java cu instrucțiuni repetitive

Instrucțiuni repetitive

În limbajul Java se utilizează uzual ca și în C++ cele trei instrucțiuni repetitive: *for*, *while* și *do-while*. Sintaxa acestor instrucțiuni și modul de utilizare în Java este la fel ca în C++. Pentru a termina mai repede o instrucțiune repetitivă se folosesc instrucțiunile *break* (la apariția ei se iese din cea mai din interior instrucțiune repetitivă) și *continue* (se trece automat la pasul următor din instrucțiunea repetitivă cea mai interioară).

Probleme rezolvate

1. Se dă n număr natural cu cel mult 5 cifre. Afișați crescător și descrescător numerele $1, 2, \dots, n$, pe rânduri diferite separate prin câte un spațiu.

Exemplu

Intrare	Ieșire
$n=10$	1 2 3 4 5 6 7 8 9 10 10 9 8 7 6 5 4 3 2 1

Soluție

```
import java.util.Scanner;

public class afisare {

    public static void main(String[] args) {

        int n,i;
```

```

Scanner cin=new Scanner(System.in);

System.out.print("n=");

n=cin.nextInt();

for(i=1;i<=n;i++)

    System.out.print(i+" ");

System.out.println();

//trecere cursorul pe rand nou

for(i=n;i>=1;i--)

    System.out.print(i+" ");

System.out.println();

}

}

```

2. Se dă n număr natural nenul cu maxim 9 cifre. Afișați suma cifrelor lui n.

Exemplu

Intrare	Ieșire
n=2034	9

Soluție

```

import java.util.Scanner;

public class cifre {

    public static void main(String[] args) {

        int n,S;

        Scanner cin=new Scanner(System.in);

        System.out.print("n=");

        n=cin.nextInt();

        S=0;

        while(n>0){

```

```

        S=S+n%10;

        n=n/10;

    }

    System.out.println(S) ;

}

}

```

3. Se dau $a < b$ numere naturale cu maxim 9 cifre. Afișați numerele prime din intervalul $[a, b]$.

Exemplu

Intrare	Ieșire
a=10 b=20	11 13 17 19

Soluție

```

import java.util.Scanner;

public class prime {

    public static void main(String[] args) {

        int n,i,j,ok,nr=0,aux;

        Scanner cin=new Scanner(System.in);

        System.out.print("n=");

        n=cin.nextInt();

        for(i=1;i<=n;i++){

            aux=i;ok=1;

            if(aux<2)

                ok=0;

            for(j=2;j<=aux/2;j++)

                if(aux%j==0){

                    ok=0;

                    break;

                }

        }
    }
}

```



```

        if (ok==1)
            nr++;
    }
    System.out.print(nr) ;
}
}

```

Probleme propuse

1. Se dă n număr natural cu cel mult 5 cifre. Afișați numerele pare din intervalul $[n, 2n]$, pe o linie separate prin câte un spațiu.

Exemplu

Intrare	Ieșire
$n=5$	6 8 10

2. Se dau a, b două cifre. Afișați a^b și numărul de cifre ale lui a^b pe linii diferite.

Exemplu

Intrare	Ieșire
$a=3$ $b=4$	81 2

3. Se dă n număr natural cu maxim 4 cifre. Afișați numărul de numere prime $\leq n$.

Exemplu

Intrare	Ieșire
$n=10$	4 (2, 3, 5, 7)

4. Se dau a, b numere naturale nenule cu cel mult 17 cifre fiecare. Verificați dacă fracția a/b este ireductibilă.

Exemplu

Intrare	Ieșire
$a=126$ $b=121$	DA

5. Se dă n număr natural cu cel mult 3 cifre. Afișați termenii din șirul lui Fibonacci în ordine crescătoare separați prin câte un spațiu mai mici sau egali cu n .

Exemplu

Intrare	Ieșire
$n=20$	1 1 2 3 5 8 13

6. Se dau a, b, c, d numere naturale nenule cu cel mult 17 cifre fiecare. Afișați suma $a/b + c/d$ în formă de fracție ireductibilă.

Exemplu

Intrare	Ieșire
$a=12$ $b=24$ $c=10$ $d=2$	$11/2$

7. Se dă o progresie aritmetică cu primul termen a , rația r și n un număr natural cu cel mult 3 cifre. a și r sunt numere naturale cu maxim 4 cifre. Afișați primii n termeni din progresia dată pe o linie, separați prin câte un spațiu.

Exemplu

Intrare	Ieșire
$a=2$ $r=3$ $n=5$	2 5 8 11 14

8. Se dau n, k numere naturale nenule cu cel mult 10 cifre fiecare. Verificați dacă n se poate scrie ca sumă de k numere naturale distincte. În caz afirmativ se va afișa o modalitate de scriere a lui n ca sumă de k numere naturale distincte.

Exemplu

Intrare	Ieșire
$n=10$ $k=3$	DA 1 4 5

3. Tablouri

Tablouri unidimensionale (vectori)

În limbajul Java lucrul cu tablouri unidimensionale (sau vectori) presupune parcurgerea a două etape: declarare și alocare de memorie.

1. Declarare:

```
Tip_componente numetablou [];
```

sau

```
Tip_componente[] numetablou;
```

2. Alocare de memorie:

```
numetablou = new Tip_componente [dimensiune];
```

Componentele tabloului unidimensional sunt:

```
numetablou[0], numetablou[1], ..., numetablou[dimensiune-1]
```

Operațiile specifice cu vectori sunt aceleași ca în C++.

Probleme rezolvate

1. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați pe rânduri diferite componentele pare, respectiv impare.

Exemplu

Intrare	Ieșire
7	2 4 4 8
1 2 3 4 4 8 5	1 3 5

Soluție

```
import java.util.Scanner;

public class vector1 {

    public static void main(String[] args) {

        int n,i,x[];

        Scanner cin=new Scanner(System.in);

        n=cin.nextInt();

        x=new int [n+1];

        for(i=1;i<=n;i++)

            x[i]=cin.nextInt();

        for(i=1;i<=n;i++)

            if(x[i]%2==0)

                System.out.print(x[i]+" ");

        System.out.println();

        for(i=1;i<=n;i++)

            if(x[i]%2==1)

                System.out.print(x[i]+" ");

    }

}
```

2. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați pe rânduri diferite maximul și minimul din vector împreună cu pozițiile lor.

Exemplu

Intrare	Ieșire
7	8
1 2 8 1 4 8 1	pozmax: 3 6
	1
	pozmin: 1 4 7

Soluție

```
import java.util.Scanner;

public class vector2 {

    public static void main(String[] args) {

        int n,i,Max,Min,x[];

        Scanner cin= new Scanner(System.in);

        n=cin.nextInt();

        x=new int[n+1];

        for(i=1;i<=n;i++)

            x[i]=cin.nextInt();

        Max=x[1];

        Min=x[1];

        for(i=2;i<=n;i++){

            if(x[i]>Max)

                Max=x[i];

            if(x[i]<Min)

                Min=x[i];

        }

        System.out.println(Max);

        System.out.print("pozmax: ");

        for(i=1;i<=n;i++)

            if(x[i]==Max)

                System.out.print(i+" ");

        System.out.println();

        System.out.println(Min);

        System.out.print("pozmin: ");
```

```

        for(i=1;i<=n;i++)

            if(x[i]==Min)

                System.out.print(i+" ");

    }

}

```

3. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați componentele vectorului ordonate crescător.

Exemplu

Intrare	Ieșire
7 1 2 8 1 4 8 1	1 1 1 2 4 8 8

Soluție

```

import java.util.Scanner;
public class vector3 {
    public static void main(String[] args) {
        int n,i,j,aux,x[];
        Scanner cin= new Scanner(System.in);
        n=cin.nextInt();
        x=new int[n+1];
        for(i=1;i<=n;i++)
            x[i]=cin.nextInt();
        for(i=1;i<n;i++)
            for(j=i+1;j<=n;j++)
                if(x[i]>x[j]){
                    aux=x[i];
                    x[i]=x[j];
                    x[j]=aux;
                }
        for(i=1;i<=n;i++)
            System.out.print(x[i]+" ");

    }

}

```

Tablouri bidimensionale

În limbajul Java lucrul cu tablouri bidimensionale presupune parcurgerea a două etape: declarare și alocare de memorie.

1. Declarare:

```
Tip_componente numetablou[][];
```

sau

```
Tip_componente[][] numetablou;
```

2. Alocare de memorie:

```
numetablou = new Tip_componente [dim1] [dim2];
```

Componentele tabloului unidimensional sunt :

```
numetablou[i][j],  $1 \leq i \leq \text{dim1} - 1$ ,  $1 \leq j \leq \text{dim2} - 1$ .
```

Operațiile specifice cu tablouri bidimensionale sunt aceleași ca în C++.

Probleme rezolvate

1. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m$, $n \leq 100$. Afișați pe rânduri diferite coloanele, care sunt ordonate crescător.

Exemplu

Intrare	Ieșire
4 5	2 5 8 9
1 2 7 3 4	3 6 8 10
7 5 9 6 2	
4 8 5 8 3	
3 9 45 10 1	

Soluție

```
import java.util.Scanner;

public class tabloul {

    public static void main(String[] args) {
        Scanner cin = new Scanner (System.in);
        int a[][] ,m,n,i,j,k,ok;
        // citirea tabloului
        m=cin.nextInt();
        n=cin.nextInt();
        a= new int[m+1][n+1];
        for (i=1;i<=m;i++)
            for (j=1;j<=n;j++)
                a[i][j]=cin.nextInt();
        for(j=1;j<=n;j++){
            //verificam daca a[1][j], a[2][j],...,a[i][j], a[k][j],
            // ..., a[m][j] sunt ordonate crescator
            ok=1;
            for(i=1;i<m;i++)
                if(a[i][j]>a[i+1][j])
                {
                    ok=0;break;
                }
            if(ok==1)
            {
                //afisam coloana j
                System.out.print("col "+j+": ");
                for(i=1;i<=m;i++)
                    System.out.print(a[i][j]+" ");
                System.out.println();
            }
        }
    }
}
```


2. Se dă un tablou pătratic de dimensiune n cu componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Ordoneați crescător fiecare linie și apoi afișați tabloul.

Exemplu

Intrare	Ieșire
4	1 2 3 7
1 2 7 3	5 6 7 9
7 5 9 6	4 5 8 8
4 8 5 8	3 9 10 45
3 9 45 10	

Soluție

```
import java.util.Scanner;
public class P2 {
public static void main(String[] args) {
    Scanner cin = new Scanner (System.in);
    int a[][] ,n,i,j,k,aux;
    //citire tabloului
    n=cin.nextInt();
    a= new int[n+1][n+1];
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++)
            a[i][j]=cin.nextInt();
    for (i=1;i<=n;i++){
        // ordonam crescator elementele
        // a[i][1],...,a[i][j]...a[i][k]..a[i][n]
        for(j=1;j<=n-1;j++)
            for(k=j+1;k<=n;k++){
                if (a[i][j]>a[i][k]){
                    aux=a[i][j];
                    a[i][j]=a[i][k];
                    a[i][k]=aux;
                }
            }
    }
    //afisarea tabloului
    for (i=1;i<=n;i++){
        for (j=1;j<=n;j++)
            System.out.print(a[i][j]+ " ");
        System.out.println();
    }
}
```

Probleme propuse

1. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați pe rânduri diferite, separate prin câte un spațiu componentele prime, respectiv neprime.

Exemplu

Intrare	Ieșire
7	2 7 13
1 2 8 7 4 8 13	1 8 4 8

2. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Verificați dacă vectorul reprezintă o mulțime.

Exemplu

Intrare 1	Ieșire 1
5	NU
2 5 3 8 3	
Intrare 2	Ieșire 2
5	DA
2 5 1 8 3	

3. Se dau doi vectori x, y cu m , respectiv n cu componente numere naturale distincte cu maxim 9 cifre fiecare. Se cere să se afișeze componentele comune lui x și y .

Exemplu

Intrare	Ieșire
4	6 8
3 6 4 8	
5	
7 8 9 1 6	

4. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați perechile de componente din vector prime între ele, pe rânduri diferite.

Exemplu

Intrare	Ieșire
5	4 5
4 5 3 8 30	4 3
	5 3
	5 8
	3 8

5. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați cea mai mare sumă ce se poate obține cu $\lfloor n/2 \rfloor$ elemente din vector.

Exemplu

Intrare	Ieșire
7	24 (8, 10, 6)
8 2 10 3 4 6 1	

6. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m, n \leq 100$. Afișați pe rânduri diferite liniile ordonate crescător.

Exemplu

Intrare	Ieșire
4 5	7 5 9 61 200
1 2 7 3 4	3 9 45 100 109
7 5 9 61 200	
4 8 5 8 3	
3 9 45 100 109	

7. Se dă un tablou pătratic de dimensiune n cu componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați pentru fiecare diagonală numele prime.

Exemplu

Intrare	Ieșire
4	DP: 5 7
1 2 7 3	DS: 3 31
7 5 9 6	
4 8 7 8	
31 9 45 10	

8. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m, n \leq 100$. Afișați pe rânduri diferite liniile ordonate crescător.

Exemplu

Intrare	Ieșire
4 5	7 5 9 61 200
1 2 7 3 4	3 9 45 100 109
7 5 9 61 200	
4 8 5 8 3	
3 9 45 100 109	

9. Se dă un tablou pătratic de dimensiune n cu componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați suma elementelor de sub diagonala principală, respectiv secundară, una sub alta.

Exemplu

Intrare	Ieșire
4	21
1 2 7 3	23
7 5 9 6	
4 8 7 8	
1 0 1 1	

10. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m, n \leq 100$. Afișați pe rânduri diferite liniile cu suma elementelor maximă.

Exemplu

Intrare	Ieșire
4 5	2 0 1 1 1
1 0 0 0 1	1 1 1 1 1
2 0 1 1 1	
0 0 0 1 0	
1 1 1 1 1	

4. Programe Java cu o singură clasă. Date membru și metode

Noțiuni introductive

Un program în limbajul Java care conține o singură clasă (clasa principală) are următoarea formă generală:

```
public class nume{  
    static tip lista date membru;  
    static tip numefunctie(parametrii){  
        ...  
        ...  
    }  
    public static void main(String[] args){  
        ...  
        ...  
    }  
}
```

Datele membru sunt variabile ce pot fi utilizate în toate metodele (funcțiile) clasei.

Observații

1. Apelul metodelor este numai prin valoare, acestea putând fi utilizate ca în limbajul C++.
2. Ordinea de scriere a metodelor și datelor membru nu are importanță.
3. Returnarea unei valori într-o metodă se realizează ca în C++, folosind instrucțiunea **return**.
4. Se pot defini într-o clasă și metode recursive.
5. Pentru o clasă datele membru sunt precum variabilele globale într-un program C++.

Probleme rezolvate cu metode nerecursive

1. Se dă n număr natural cu cel mult 9 cifre. Afișați numerele palindrom mai mici sau egale cu n. Un număr natural este palindrom dacă citit de la stânga la dreapta sau invers se obține același număr.

Exemplu

Intrare	Ieșire
n=25	0 1 2 3 4 5 6 7 8 9 11 22

Soluție

Vom crea o clasă cu n - dată membru și două metode: main și o o metodă pentru verificarea condiției de număr palindrom.

```
import java.util.Scanner;

public class program1 {

    static int n;

    static Scanner cin=new Scanner(System.in);

    static boolean palindrom(int k)

    {

        int aux,c,inv;

        inv=0;

        aux=k;
```

```

        while (aux>0)
        {
            c=aux%10;
            inv=inv*10+c;
            aux/=10;
        }

        if (inv==k)
            return true;
        return false;
    }

    public static void main(String[] args) {
        System.out.print("n=");
        n=cin.nextInt();
        int i;
        for (i=0;i<=n;i++)
            if (palindrom(i))
                System.out.print(i+" ");
    }
}

```

2. Se dau două numere întregi a și b cu maxim 9 cifre fiecare, $a < b$. Afișați numerele naturale din intervalul $[a, b]$, care sunt o putere a lui 2 în ordine crescătoare.

Exemplu

Intrare a=7 b=50	Ieșire 8 16 32
-------------------------------	--------------------------

Soluție

Vom crea o clasă în care a și b sunt date membru și două metode: main și o metodă pentru verificarea condiției de număr putere a lui 2, pentru un număr dat ca parametru.

```
import java.util.Scanner;

public class program2 {

    static int a,b;

    static Scanner cin = new Scanner(System.in);

    static boolean putere(int k){

        while (k%2==0)

            k/=2;

        if (k==1)

            return true;

        return false;

    }

    public static void main(String[] args) {

        System.out.print("a=");

        a=cin.nextInt();

        System.out.print("b=");

        b=cin.nextInt();

        int i;

        for (i=a;i<=b;i++)

            if(putere(i))

                System.out.print(i+" ");

    }

}
```


3. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10^9 , $1 \leq n \leq 100$. Afișați componentele vectorului care au prima și ultima cifră egale pe o linie, respectiv celelalte componente pe o altă linie.

Exemplu

Intrare	Ieșire
7	434 89218 7 9009
123 434 56 89218 7 12 9009	123 56 12

Soluție

Vom utiliza o metodă pentru citirea vectorului și o metodă pentru verificarea condiției legată de prima și ultima cifră. Dimensiunea vectorului, vectorul și obiectul `cin` folosit la citirea datelor vor fi date membru.

```
import java.util.Scanner;

public class program3 {

    static int n, x[];

    static Scanner cin = new Scanner(System.in);

    public static void cit(){

        int i;

        n=cin.nextInt();

        x= new int[n+1];

        for (i=1;i<=n;i++)

            x[i]=cin.nextInt();

    }

    static boolean verific(int k){

        int u, p;

        u=k%10;

        p=k;

        while (p>9)

            p/=10;
```

```

        if (p==u)
            return true;
        return false;
    }

    public static void main(String[] args) {
        cit();
        int i;
        for (i=1;i<=n;i++)
            if (verif(x[i]))
                System.out.print(x[i]+" ");
    }
}

```

4. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m$, $n \leq 100$. Afișați pe rânduri diferite numerele prime de pe fiecare linie.

Exemplu

Intrare	Ieșire
4 5	2 7 3
1 2 7 3 4	7 5 2
7 5 9 6 2	5 3
4 8 5 8 3	3
3 9 45 10 1	

Soluție

Vom utiliza o metodă pentru citirea tabloului și o metodă pentru verificarea condiției de număr prim. Dimensiunile tabloului, tabloul și obiectul **cin** folosit la citirea datelor vor fi date membru.

```

import java.util.Scanner;

public class program4 {

    static Scanner cin= new Scanner(System.in);

```

```

static int n,m,a[][];

static void cit () {
    n=cin.nextInt();
    m=cin.nextInt();
    a= new int[m+1][n+1];
    int i,j;
    for (i=1;i<=m;i++)
        for (j=1;j<=n;j++)
            a[i][j]=cin.nextInt();
}

static boolean prim(int k) {
    int d;
    if(k<2)
        return false;
    for (d=2;d<=k/2;d++)
        if (k%d==0)
            return false;
    return true;
}

public static void main(String[] args) {
    cit();
    int i,j;
    for (i=1;i<=m;i++){
        for (j=1;j<=n;j++)
            if (prim(a[i][j]))
                System.out.print(a[i][j]+" ");
        System.out.println();
    }
}
}

```

Probleme rezolvate folosind metode recursive

1. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10^9 , $1 \leq n \leq 100$. Verificați dacă numerele $1, 2, \dots, n^2$ se găsesc în vector. În caz afirmativ se vor afișa împreună cu poziția în vector.

Exemplu

Intrare	Ieșire
7 123 434 5 89218 7 22 9009	5 (3) 7 (5) 22 (6)

Soluție

Vom utiliza o metodă pentru citirea vectorului și o metodă recursivă pentru afișarea numerelor cerute. Dimensiunea vectorului, vectorul și obiectul **cin** folosit la citirea datelor vor fi date membru.

```
import java.util.Scanner;

public class program5 {

    static Scanner cin = new Scanner(System.in);

    static int n,x[];

    static void cit(){

        n= cin.nextInt();

        x= new int[n+1];

        int i;

        for (i=1;i<=n;i++)

            x[i]=cin.nextInt();

    }
```

```

static void verif(int k){
    if (k<=n){
        if(x[k]<=n*n)
            System.out.print(x[k]+" (" +k+" ) ");
        verif(k+1);
    }
}

public static void main(String[] args) {
    cit();
    verif(1);
}
}

```

2. Se dă n număr natural mi mic sau egal cu 15. Afișați valoarea sumei $1! + 2! + \dots + n!$.

Exemplu

Intrare	Ieșire
n=4	33

Soluție

```

import java.util.Scanner;

public class program6 {
    static int n;

    static int factorial(int k){
        if (k==1)
            return 1;
        return factorial(k-1)*k;
    }
}

```

```

public static void main(String[] args) {
    int i,s=0;
    Scanner cin = new Scanner(System.in);
    System.out.print("n=");
    n=cin.nextInt();
    for(i=1;i<=n;i++)
        s+=factorial(i);
    System.out.print(s);
}
}

```

3. Se dă n număr natural mi mic sau egal cu 15. Afișați permutările mulțimii {1, 2, ..., n}.

Exemplu

Intrare	Ieșire
n=3	1 2 3 1 3 2 2 1 3 2 3 1 3 1 2 3 2 1

Soluție

Pentru rezolvarea problemei folosim metoda backtracking în variantă recursivă.

```

import java.util.Scanner;

public class program7 {
    static Scanner cin = new Scanner(System.in);
    static int n,v[]=new int[20];
    static void afis(){
        int i; // afisarea permutarii curente din v
        for (i=1;i<=n;i++)

```

```

        System.out.print(v[i]+" ");
        System.out.println();
    }
    static boolean verif(int k){
        int i;
        // verificam daca v[k] este diferit de v[1], ..., v[k-1]
        for (i=1;i<k;i++)
            if(v[k]==v[i])
                return false;
        return true;
    }
    static void back(int k){
        int i;
        if(k>n)
            afis();
        else // generarea valorilor posibile pentru v[k]
            for(i=1;i<=n;i++)
            {
                v[k]=i;
                if (verif(k))
                    back(k+1);
            }
    }
    public static void main(String[] args) {
        n=cin.nextInt();
        back(1);
    }
}

```

Probleme propuse

1. Se dă n număr natural cu cel mult 5 cifre. Afișați numerele prime din intervalul $[n, 2n]$.

Exemplu

Intrare $n=6$	Ieșire 7 11
-------------------------	-----------------------

2. Se dau a și b numere naturale cu maxim 9 cifre fiecare. Pentru fiecare număr a, b , afișați suma și produsul cifrelor nenule.

Exemplu

Intrare $a=320$ $b=145$	Ieșire 5 6 10 20
--------------------------------------	-------------------------------

3. Se dau a, b, c numere întregi cu cel mult 17 cifre fiecare, $a \leq b$. Afișați valoarea sumei $p(a)! + p(a+1)! + \dots + p(b)!$, unde $p(k)$ reprezintă prima cifră a lui k , iar $k! = 1 \cdot 2 \cdot \dots \cdot k$.

Exemplu

Intrare $a=198$ $b=202$	Ieșire 8
--------------------------------------	--------------------

4. Se dă un vector cu n componente numere naturale. Câte componente termeni din șirul lui Fibonacci (1, 1, 2, 3, 5, 8, 13, ...) sunt în vector?

Exemplu

Intrare 5 4 8 11 3 1	Ieșire 3
-----------------------------------	--------------------

5. Se dă un vector cu n componente numere naturale. Afișați cel mai mare divizor comun pentru toate componentele vectorului.

Exemplu

Intrare 4 10 100 250 50	Ieșire 5
--------------------------------------	--------------------

6. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m, n \leq 100$. Afișați pe rânduri diferite liniile care conțin un număr maxim de numere prime.

Exemplu

Intrare 4 5 1 2 7 3 4 7 5 9 6 2 4 8 5 8 3 3 9 45 10 1	Ieșire 1 2 7 3 4 7 5 9 6 2
---	---

7. Se dă un tablou pătratic de dimensiune n cu componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați pentru fiecare linie cel mai mare, respectiv cel mai mic număr separate prin câte un spațiu.

Exemplu

Intrare 4 1 2 7 3 7 5 9 6 4 8 5 8 3 9 45 1	Ieșire 7 1 9 5 8 4 45 1
--	--

8. Se dă n număr natural mai mic sau egal cu 17. Afișați toate submulțimile mulțimii $\{1, 2, \dots, n\}$.

Exemplu

Intrare 4	Ieșire { } {1} {1 2} {1 2 3} {1 3} {2} {2 3} {3}
---------------------	--

9. Se dă n număr natural mai mic sau egal cu 10. Afișați toate numerele cu exact n cifre, toate impare, astfel încât să nu existe două cifre egale una lângă alta.

Exemplu

Intrare	Ieșire
4	1313 1315 1317 ...

10. Se dă n număr natural mai mic sau egal cu 20. Afișați toate modalitățile de scriere a lui n ca sumă de numere impare.

Exemplu

Intrare	Ieșire
4	1 1 1 1 1 3

5. Principiile Programării Orientate pe Obiecte

Noțiuni fundamentale

Principiile Programării Orientate pe Obiecte sunt următoarele:

1. Încapsulare;
2. Moștenire;
3. Polimorfism.

1. *Încapsularea* se referă la posibilitatea de a crea noțiuni ce conțin date și metode (clase). O clasă în Java are forma generală:

```
class nume{
    // date membru
    int a, b;
    static int x, y;
    ...
    // metode
    ... main (String[] args){
        ...
    }
}
```

2. *Moștenirea* se referă la posibilitatea de a construi clase pornind de la alte clase deja definite (pe care le completăm cu noi metode și date membru, pentru a obține clase noi).

3. *Polimorfismul* se poate caracteriza prin posibilitatea ca o metodă să se redefinească în procesul de moștenire.

Noțiunea de obiect

Pentru o clasă dată putem să asociem mai multe variabile (numite obiecte), așa cum se lucrează cu **struct** în C++.

Observații

1. Obiectul este o instanță a clasei (variabilă de "tip" clasă)
2. Crearea obiectelor se realizează astfel:

```
numeclasă ob;
```

```
ob = new numeclasă();
```

sau

```
numeclasă ob = new numeclasă();
```

3. Accesul la datele și metodele unui obiect ob:

```
ob.dată_membru
```

```
ob.metodă
```

Exemplu de creare și utilizare a obiectelor

```
class exemplu{
    int x, y;
    void afis(){
        System.out.println(x + " " + y);
    }
}

public class utilizare{
    public static void main(String[] args){
```

```

        exemplu o1, o2;
        o1 = new exemplu();
        o2 = new exemplu();
        o1.x = 10;  o1.y = 25;
        o2.x = 12;  o2.y = 101;
        System.out.println("Date obiect o1: ");
        o1.afis();
        System.out.println("Date obiect o2: ");
        o2.afis();
    }
}

```

Noțiunea de constructor

Constructorul este o metodă care are același nume cu numele clasei, nu are tip și este folosit pentru inițializarea datelor membru din clasă.

Exemplu

```

class punct{
    int x,y;
    punct(int x1,int y1){
        x=x1;y=y1;
    }
    punct(int x1){
        x=x1;
    }
    void afis(){
        System.out.println("P("+x+", "+y+") ");
    }
}

public class utilizare1 {
    public static void main(String[] args) {
        punct p1;
        p1=new punct(10,50);
        punct p2;
        p2=new punct(100);
        p1.afis();p2.afis();
    }
}

```

Observații

1. O clasă poate avea mai mulți constructori. Diferența dintre ei constă în faptul că au un număr diferit de parametri sau tipul parametrilor este diferit.
2. Dacă o clasă nu are un constructor definit, atunci compilatorul folosește un constructor implicit prin care datele membru sunt inițializate implicit (de exemplu datele numerice sunt inițializate cu 0).

Cuvântul cheie `this`

Pentru a face referire la datele obiectului curent se folosește cuvântul **`this`**, într-o construcție de forma **`this.data_membru`**.

Exemplu

```
class fractie{
    int a, b;
    fractie(int a, int b){
        this.a = a;
// this.a este data membru a, iar a din dreapta
// atribuirii este parametrul a din metoda constructor
        this.b = b;
    }
}
```

Date și metode statice

Cuvântul cheie **`static`** semnifică faptul că pentru toate datele și metodele obiectelor clasei se alocă o singură zonă de memorie. Astfel, accesul la datele și metodele clasei se poate face în mai multe moduri.

În cazul static accesul la date și metode se poate face astfel:

```
obiect.data
obiect.metoda
sau
```

```
numeclasă.data
numeclasă.metoda
```

Exemplu

```
class numar{
    static int n=1;
    static int pp(){
        return n*n;
    }
    static int cub(){
        return n*n*n;
    }
}

public class utilizare2 {
    public static void main(String[] args) {
        numar x;
        x=new numar();
        x.n=10;
        System.out.println(x.n+" "+numar.n);
        System.out.println(x.pp()+" "+numar.pp());
        System.out.println(x.cub()+" "+numar.cub());
    }
}
```

Datele și metodele nestatice se caracterizează prin faptul că pentru fiecare componentă a unui obiect al clasei se alocă o câte o zonă de memorie. Atributul nestatic este implicit.

Referințe către obiecte

Putem să folosim obiecte ca parametri pentru metode:

```
tip nume_metoda (nume_clasa1 ob1, nume_clasa1 ob2, ...) {  
    ...  
}
```

Putem să utilizăm metode care returnează obiecte:

```
nume_clasa nume_metoda (...) {  
    ...  
    return obiect ;  
}
```

Putem avea tablouri cu componente obiecte.

Exemplu

```
class C {  
    ...  
}  
...  
C x[];  
x = new C[100];  
for(i=0; i<100; i++)  
    x[i]=new C(...);  
  
//Componentele din obiectul x[i] le accesăm prin  
//x[i].data_membru, respectiv x[i].metoda  
...
```


Probleme rezolvate folosind obiecte

1. Determinarea coordonatelor mijlocului unui segment. Segmentul este dat prin coordonatele capetelor.

Soluție

```
class punct{
double x,y;

    punct(double x,double y){
        this.x=x;
        this.y=y;
    }
    void afis(){
        System.out.println("(" +x+" "+y+"");
    }
    punct mijloc(punct P){
        punct Q;
        Q=new punct(0,0);
        Q.x=(x+P.x)/2;
        Q.y=(y+P.y)/2;
        return Q;
    }
}

public class utilizare3 {
    public static void main(String[] args) {
        punct P,Q,R;
        P=new punct(10,20);
```

```

        Q=new punct(100,50) ;

        R=P.mijloc(Q) ;

        R.afis() ;

    }

}

```

2. Calculați suma și produsul a n fracții. Frațiile sunt date prin numărător și numitor.

Exemplu

Intrare	Ieșire
a=7 b=50	8 16 32

Soluție

```

import java.util.Scanner;

class fractie {

    int a,b;

    fractie(int a,int b){

        this.a=a;

        this.b=b;

    }

    void afis(){

        System.out.println(a+"/"+b) ;

    }

    void simplificare(){

        int x,y;

        x=a;y=b;

        while(x!=y)

            if(x>y)

```

```

        x-=y;

        else

        y-=x;

        a/=x;

        b/=x;

    }

void adun(fractie F){

    int x,y;

    x=a*F.b+b*F.a;

    y=b*F.b;

    a=x;

    b=y;

    simplificare();

}

void prod(fractie F){

    int x,y;

    x=a*F.a;

    y=b*F.b;

    a=x;b=y;

    simplificare();

}

}

public class utilizare4 {

    public static void main(String[] args) {

        int n;

        Scanner cin = new Scanner(System.in);

```

```

n = cin.nextInt();
fractie f[];
f=new fractie[n+1];
int i,x,y;//citirea fractiilor
for (i=1;i<=n;i++){
    x = cin.nextInt();
    y = cin.nextInt();
    f[i] = new fractie(x,y);
}
fractie s,p;
//initializarea lui s cu fractia 0/1
s = new fractie(0,1);
//initializarea lui p cu fractia 1/1
p = new fractie(1,1);
for (i=1;i<=n;i++)
    f[i].simplificare(); //simplificarea fiecărei
fractii
for (i=1;i<=n;i++){
    s.adun(f[i]);
    p.prod(f[i]);
}
System.out.println("Suma= ");
s.afis();
System.out.println("Prod= ");
p.afis();
}
}

```

3. Un interval închis $[a, b]$ este dat prin capetele sale $a, b, a < b$. Creați clasa interval cu: a, b - date membru și metodele:

- constructor

- afișare

- lungime

a) Creați un obiect și apelați metodele lui.

b) Citiți datele pentru n intervale. Afișați pentru fiecare interval datele membru.

c) Afișați intervalele ordonate crescător după lungime.

d) Pentru k dat, afișați intervalele cu lungimea egală cu k .

Soluție

```
import java.util.Scanner;

class interval{
    int a,b;
    interval(int a,int b){
        this.a=a;
        this.b=b;
    }
    void afis(){
        System.out.println("["+a+", "+b+"]");
    }
    int lung(){
        return b-a;
    }
}

public class utilizare5 {
    public static void main(String[] args) {
        int n,a,b,i,k,j;
        interval x[]=new interval[101],aux;
```

```

Scanner cin=new Scanner(System.in);
n=cin.nextInt();
k=cin.nextInt();
for(i=1;i<=n;i++){
    a=cin.nextInt();
    b=cin.nextInt();
    x[i]=new interval(a,b);
}
//a)+b)
for(i=1;i<=n;i++)
    x[i].afis();
//c)
for(i=1;i<n;i++)
    for(j=i+1;j<=n;j++)
        if(x[i].lung()>x[j].lung()){
            aux=x[i];
            x[i]=x[j];
            x[j]=aux;
        }
for(i=1;i<=n;i++)
    x[i].afis();
//d)
for(i=1;i<=n;i++)
    if(x[i].lung()==k)
        x[i].afis();
}
}

```

Probleme propuse

1. Creați o clasă dreptunghi cu:

- date membru: a,b - lungime și lățime

- metode:

a) constructor

b) afișare

c) arie

d) perimetru

Verificați clasa dreptunghi pentru un obiect în clasa principală cu numele test1

2. Citiți datele pentru n dreptunghiuri folosind un vector cu componente obiecte ale clasei dreptunghi (definită la problema 1). Afișați aria și perimetrul pentru fiecare dreptunghi.

Exemplu

Intrare	Ieșire
2	200 60
10 20	300 206
100 3	

3. Afișați dreptunghiurile (lungimea și lățimea lor) de la problema 2 ordonate descrescător după arie.

Exemplu

Intrare	Ieșire
2	100 3
10 20	10 20
100 3	

4. Pentru dreptunghiurile de la problema 2 afișați dreptunghiurile cu perimetrul maxim.

Exemplu

Intrare	Ieșire
3	100 3
10 20	102 1
100 3	
102 1	

5. Folosind clasa fracție definită în a doua problemă rezolvată, afișați diferența și raportul a două fracții în formă ireductibilă.

Exemplu

Intrare	Ieșire
3	1/4
10 20	2/1
100 400	

6. a) Se cere să se creeze o clasă cu numele patrat, cu o singură dată membru - lungimea laturii notată cu a și metodele:

- constructor
- perimetru
- arie

b) În clasa testpatrat definiți un obiect al clasei patrat și verificați toate metodele.

c) Citiți datele pentru n pătrate și afișați pătratele crescător după perimetru. Se va afișa: lungimea laturii, perimetrul și aria.

6. Clasa Math, clasa String și clase înfășurătoare

Pachetul `java.util` conține clase specializate pe diverse prelucrări, cum sunt **Math** pentru date numerice și **String** pentru șiruri de caractere.

Clasa Math

Clasa **Math** este folosită pentru a prelucra date numerice, precum biblioteca `math.h` din C++. Aceasta este o clasă statică, lucru care presupune accesarea metodelor prin construcții de forma **Math.numemetoda(...)**.

Câteva metode utile din clasa Math:

Math.sqrt(x)	==>	$\sqrt{x}$
Math.pow(a,b)	==>	a^b
Math.abs(x)	==>	$ x $
Math.ceil(x)	==>	cel mai mic număr întreg $\geq x$
Math.floor(x)	==>	cel mai mare număr întreg $\leq x$
Math.random()	==>	returnează un număr real din $[0, 1)$

Observație

Pentru a determina aleator un număr natural din intervalul $[0, n)$, n număr natural, folosim secvența:

```
int k;
k=Math.round(Math.random()*n);
```

Probleme rezolvate

1. Definiți clasa segment pentru a memora un segment dat prin coordonatele capetelor, apoi:

- citiți datele pentru n segmente;
- afișați numărul de segmente de lungime minimă;
- afișați segmentele (coordonatele capetelor) crescător, după lungime.

Clasa va conține coordonatele capetelor de segmentului: x1,x2,y1, y2, iar ca metode: constructor, afis – pentru afișarea coordonatelor capetelor segmentului și lungime – pentru lungimea segmentului.

Exemplu

Intrare	Ieșire
3	nr de segm min= 1
0 0 -10 0	0 0 5 0
0 0 5 0	0 0 -10 0
10 0 10 10	10 0 10 10

Soluție

```
import java.util.Scanner;

class segment{

    int x1,x2,y1,y2;

    segment(int x1, int x2, int y1, int y2){

        this.x1=x1;

        this.y1=y1;

        this.x2=x2;

        this.y2=y2;

    }

    void afis(){

        System.out.println("(" +x1+", "+x2+" ) (" +y1+", "+y2+" )");

    }

    double lungime(){

        return Math.sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));

    }

}
```

```

public class test1 {
    public static void main(String[] args){
int n,x1,y1,x2,y2,i,j;

        segment s[]=new segment[100];

        Scanner cin = new Scanner(System.in);
        n=cin.nextInt();
        for(i=1;i<=n;i++){
            x1=cin.nextInt();
            y1=cin.nextInt();
            x2=cin.nextInt();
            y2=cin.nextInt();
            s[i]=new segment(x1,y1,x2,y2);
        }

double Min=s[1].lungime();
int nr_min=1;
for(i=2;i<=n;i++)
    if(Min>s[i].lungime())
        Min=s[i].lungime();
for(i=2;i<=n;i++)
    if(Min==s[i].lungime())
        nr_min++;
System.out.println("nr de segm min="+nr_min);
segment aux;
for(i=1;i<n;i++)
    for(j=i+1;j<=n;j++)
        if(s[i].lungime()>s[j].lungime()){

```

```

        aux=s[i];

        s[i]=s[j];

        s[j]=aux;

    }

    for (i=1;i<=n;i++)

        s[i].afis();

    System.out.println();

}

}

```

2. Se consideră triunghiuri date prin lungimile laturilor: a, b, c. Definiți o clasă cu numele triunghi care să conțină:

- a, b, c - date membru

- metode:

- constructor

- perimetru

- arie

- afișare

Citiți datele despre n triunghiuri și afișați triunghiurile în ordinea descrescătoare a ariei (lungimea laturii și aria).

Exemplu

Intrare	Ieșire
4	(3, 3, 3) 3.89
3 4 5	(3, 4, 5) 6
3 3 3	(4, 4, 4) 6.92
6 10 8	(6, 10, 8) 24
4 4 4	

Soluție

```
import java.util.Scanner;

class triunghi{
    int a,b,c;

    triunghi(int a,int b,int c){
        this.a=a;
        this.b=b;
        this.c=c;
    }

    int perimetru(){
        return a+b+c;
    }

    double arie(){
        double p;
        p=perimetru()/2.0;
        return Math.sqrt(p*(p-a)*(p-b)*(p-c));
    }

    void afisare(){
        System.out.println("(" +a+" "+b+" "+c+" ") +arie());
    }
}

public class Pooap1 {
    public static void main(String[] args) {
        int n,i,j,a,b,c;

        triunghi x[]=new triunghi[101],aux;

        Scanner cin=new Scanner(System.in);
```

```

n=cin.nextInt();
for (i=1;i<=n;i++) {
    a=cin.nextInt();
    b=cin.nextInt();
    c=cin.nextInt();
    x[i]=new triunghi(a,b,c);
}
for (i=1;i<n;i++)
    for (j=i+1;j<=n;j++)
        if (x[i].arie()<x[j].arie()) {
            aux=x[i];
            x[i]=x[j];
            x[j]=aux;
        }
for (i=1;i<=n;i++)
    x[i].afisare();
}
}

```

3. Un paralelipiped dreptunghic este caracterizat prin: a, b - lungimile laturilor bazei și c - înălțimea.

Creați o clasă paralelipiped cu datele: a, b, c și metode pentru:

- inițializarea datelor membru
- afișare
- arie laterală
- arie totală
- volum

a) Citiți date pentru un obiect și folosiți metodele lui.

b) Citiți date pentru n paralelipede dreptunghice și afișați pentru fiecare aria laterală, totală și volumul.

c) Afișați datele pentru paralelipedul dreptunghic cu volum maxim.

Soluție

```
import java.util.Scanner;

class paralelipiped{
    int a,b,c;

    paralelipiped(int a,int b,int c){
        this.a=a;
        this.b=b;
        this.c=c;
    }

    int arielat(){
        return 2*c*(a+b);
    }

    int arietot(){
        return 2*(a*b+b*c+c*a);
    }

    int volum(){
        return a*b*c;
    }
}

public class Pooap2 {
    public static void main(String[] args) {
        int i,a,b,c,n,Max=0;
```

```

    paralelipiped x[]=new paralelipiped[101];
    Scanner cin=new Scanner(System.in);
    n=cin.nextInt();
    for(i=1;i<=n;i++){
        a=cin.nextInt();
        b=cin.nextInt();
        c=cin.nextInt();
        x[i]=new paralelipiped(a,b,c);
    }
    // a) + b)
for(i=1;i<=n;i++)
System.out.println(x[i].arielat()+" "+x[i].arietot()+"
"+x[i].volum());
    // c)
    for(i=1;i<=n;i++)
        Max=Math.max(Max,x[i].volum());
for(i=1;i<=n;i++)
    if(Max==x[i].volum())
System.out.println(x[i].a+" "+x[i].b+" "+x[i].c+"
"+x[i].volum());
    }
}

```

4. Definiți clasa complex pentru a memora un număr complex, care să conțină metodele: constructor, afis - pentru afișarea numărului complex, modul - pentru calculul modului, adun și prod pentru suma și produsul numărului curent cu cel dat ca parametru în metodă. Se dau n numere complexe. Afișați:

a) numerele citite;

- b) suma celor n numere date;
- c) produsul celor n numere date;
- d) numerele date ordonate crescător după modul.

Soluție

```
import java.util.Scanner;

class complex{
    int x,y;

    complex(int x,int y){
        this.x=x;
        this.y=y;
    }

    void afis(){
        System.out.print(x+"");
        if(y<0)
            System.out.println("(" +y+"") *i");
        else System.out.println(y+"*i");
    }

    double modul(){
        return Math.sqrt(x*x+y*y);
    }

    void adun(complex z){
        x+=z.x;
        y+=z.y;
    }
}
```

```

void prod(complex z){
int a,b;

    a=x*z.x-y*z.y;

    b=x*z.y+y*z.x;

    x=a;

    y=b;

}

}

public class testcomplex {

    public static void main(String[] args) {

        Scanner cin= new Scanner(System.in);

int n,x,y;

        n=cin.nextInt();

int i;

        complex v[];

v=new complex[n+1];

        for (i=1;i<=n;i++){

            x=cin.nextInt();

            y=cin.nextInt();

            v[i]= new complex(x,y);

        }

        for(i=1;i<=n;i++)

            v[i].afis();

        complex s,p;

s=new complex(0,0);

p=new complex(1,0);

```

```

        for(i=1;i<=n;i++)
        {
            s.adun(v[i]);
            p.prod(v[i]);
        }
System.out.print("Suma=");
s.afis();
System.out.print("Produs=");
p.afis();

        //ordonarea crescatoaredupa modul
        complex aux;
int j;
for(i=1;i<n;i++)
    for(j=i+1;j<=n;j++)
        if(v[i].modul()>v[j].modul())
            {
                aux=v[i];
                v[i]=v[j];
                v[j]=aux;
            }
System.out.println("Nr. complexe ordonate dupa modul");
        for(i=1;i<=n;i++)
            v[i].afis();
    }
}

```

Clasa String

Clasa **String** este folosită pentru a prelucra și memora șiruri de caractere (stringuri). Aceasta conține următorii constructori:

- **String()** => inițializează data membru a obiectului cu șirul de caractere vid.

Exemplu

```
String s = new String();  
s = "UPIT";  
System.out.println(s);
```

- **String("sir de caractere")** => inițializează data membru a obiectului cu șirul de caractere specificat între ghilimele.

Exemplu

```
String s = new String ("UPIT");
```

Concatenarea (lipirea) a două stringuri se realizează cu operatorul +.

Exemplu

```
String s = new String("Vasile");  
String t = new String("Popa");  
s = s + " " + t;    => s reține șirul de caractere "Vasile Popa".
```

Citirea unui șir de caractere de la tastatură presupune folosirea clasei **Scanner** și metoda **cin.nextLine()**, care returnează șirul de caractere introdus până la enter.

Câteva metode din clasa **String** utilizate frecvent:

1. **length()** => returnează numărul de caractere din obiectul curent

Exemplu

```
int n;  
  
String s = new String ("UPIT");  
  
n = s.length();           =>  n=4
```

2. **charAt(indice)** => returnează caracterul de pe poziția *indice* din data membru a obiectului curent, unde indice poate fi 0,1,...,lung-1, lung fiind numărul de caractere din data membru a obiectului curent.

Exemplu

```
String s = new String ("UPIT");  
  
char c=s.charAt(2);       =>c=' I '
```

3. **compareTo(s)** =>returneaza 0, dacă obiectul curent are șirul de caractere egal cu cel din s, o valoare strict pozitivă, dacă obiectul curent are șirul de caractere mai mic decât cel din s, respectiv o valoare strict negativă, dacă obiectul curent are șirul de caractere mai mic decât cel din s.

Exemplu

```
String t = new String("ABCDE");  
  
int n = t.compareTo("ABCPQRS");    =>  n<0
```

4. **substring(k,l)** => returnează secvență ce începe pe poziția k și se termină pe poziția l-1

Exemplu:

```
String s = new String("ABCDEFGH");  
  
String t=s.substring(2,6);          =>  t="CDEF"
```

5. **indexOf(s)** => returnează indicele de unde începe prima apariție a șirului de caractere din s în obiectul curent ca secvență de caractere. Dacă nu există, metoda returnează -1.

Exemplu:

```
String t = new String ("abcxyzabcxyzabc");  
  
int n = t.indexOf("xyz");           => n = 3  
  
int m = t.indexOf("abcd");          => m = -1
```

Probleme rezolvate

1. Se dă un cuvânt care se termină cu enter. Verificați dacă cuvântul este palindrom (citit de la stânga la dreapta sau de la dreapta la stânga se obține același cuvânt). Se va afișa DA în caz afirmativ, respectiv NU în caz negativ.

Exemplu

Intrare	Ieșire
radar	DA

Soluție

```
import java.util.Scanner;  
  
public class P1 {  
  
    static boolean cuvant_palindrom(String s) {  
        if(s.isEmpty())//verifica daca s contine sirul vid  
            return true;  
        else{  
            int i,l;  
            l=s.length();  
            for(i=0;i<=l/2;i++)  
                if(s.charAt(i)!=s.charAt(l-i-1))  
                    return false;  
        }  
    }  
}
```

```

        return true;
    }
}

public static void main(String[] args) {
    Scanner cin = new Scanner(System.in);
    String s = new String();
    s = cin.nextLine();
    if(cuvant_palindrom(s))
        System.out.println("DA");
    else
        System.out.println("NU");
}
}

```

2. Se dau două cuvinte, unul sub altul, care se termină cu enter. Verificați dacă cele două cuvinte se termină cu aceeași literă.

Exemplu

Intrare	Ieșire
Matematica	DA
Informatica	

Soluție

```

import java.util.Scanner;

public class P2 {
    static String s1,s2;

    public static void main(String[] args) {
        Scanner cin=new Scanner(System.in);

        s1=cin.nextLine();

        s2=cin.nextLine();
    }
}

```

```

if (s1.charAt(s1.length()-1) != s2.charAt(s2.length()-1))
    System.out.println("NU");
else
    System.out.println("DA");
}
}

```

3. Se dau n cuvinte, unul sub altul, care se termină cu enter. Afișați cuvintele în ordine alfabetică.

Exemplu

Intrare	Ieșire
3	Informatica
Matematica	Geometrie
Informatica	Matematica
Geometrie	

Soluție

```

import java.util.Scanner;

public class P3 {

    public static void main(String[] args){

        Scanner cin=new Scanner(System.in);

        String a[]=new String[100],aux;

        int n;

        n = cin.nextInt();

        cin.nextLine();

        int i,j;

        for(i=1;i<=n;i++)

            a[i]=cin.nextLine();

        for(i=1;i<n;i++)

```



```

        for (j=i+1; j<=n; j++)
            if (a[j].compareTo(a[i])<0) {
                aux=a[i];
                a[i]=a[j];
                a[j]=aux;
            }
        for (i=1; i<=n; i++)
            System.out.println(a[i]);
    }
}

```

4. Se dau două cuvinte, unul sub altul, care se termină cu enter. Verificați dacă al doilea cuvânt se găsește în primul ca secvență de caractere.

Exemplu

Intrare	Ieșire
Matematica	DA
tematic	

Soluție

```

import java.util.Scanner;

public class P4 {
    public static void main(String[] args){
        String s,t;
        Scanner cin = new Scanner(System.in);
        s = cin.nextLine();
        t = cin.nextLine();
        if(s.indexOf(t)>=0)
            System.out.println("DA");
        else
            System.out.println("NU");
    }
}

```

Clase înfășurătoare

Clasa înfășurătoare este o clasă asociată unui tip de date. Aceasta este utilizată pentru a realiza operații cu tipul de date respectiv prin intermediul metodelor.

Clase înfășurătoare utilizate frecvent:

1) **int** -> **Integer**

2) **float** -> **Float**

3) **long** -> **Long**

4) **char** -> **Character**

5) **boolean** -> **Boolean**

6) **double** -> **Double**

Clasele înfășurătoare au un constructor pentru inițializarea datei membru cu o valoare de tipul respectiv.

Fiecare clasă înfășurătoare conține o metodă de conversie a unui șir de caractere la tipul corespunzător.

Exemple

```
int x = 2019;
```

```
Integer o1 = new Integer(x);
```

```
Float o2 = new Float(3.14);
```

```
Character o3 = new Character('a');
```

Observație

Pentru a obține valoarea dintr-un obiect al unei clase înfășurătoare utilizăm metodele:

- **intValue()**

- `doubleValue()`
- `charValue()`
- `floatValue()`
- `booleanValue()`

Exemplu

```
int a = o1.intValue();  
float b = o2.floatValue();  
char c = o3.charValue();
```

Observație

Pentru a converti un șir de caractere s într-o valoare numerică folosim metodele:

- `parseInt(s)`
- `parseDouble(s)`
- `parseFloat(s)`
- `parseLong(s)`

Exemplu:

```
int x = parseInt("2019");  
float y = parseFloat("3.14");  
long z = parseLong("12345678910");
```

Probleme rezolvate

1. Se dau două numere naturale pe rânduri diferite. Se cere să se afișeze pentru fiecare număr numărul de cifre și apoi suma acestor numere.

Exemplu

Intrare	Ieșire
2019	4 3
121	2140

Soluție

```
import java.util.Scanner;

public class Infasuratoare1 {

    public static void main(String[] args) {

        Scanner cin = new Scanner(System.in);

        String s1, s2;

        s1 = cin.nextLine();

        s2 = cin.nextLine();

        System.out.println(s1.length() + " " + s2.length());

        int a1, b1;

        Integer o1, o2;

        o1 = new Integer(s1);

        o2 = new Integer(s2);

        a1 = o1.intValue();

        b1 = o2.intValue();

        System.out.println(a1+b1);

    }

}
```

2. Se dă n număr natural. Folosind clase înfășurătoare, afișați prima cifră a lui n și apoi suma cifrelor. Nu se vor utiliza împărțiri!
Exemplu

Intrare	Ieșire
2019	2 12

Soluție

```
import java.util.Scanner;

public class Infasuratoare2 {

    public static void main(String[] args) {

        Scanner cin=new Scanner(System.in);

        int s=0,i,l;

        String a;

        a=cin.nextLine();

        System.out.println(a.charAt(0));

        l=a.length();

        for(i=0;i<l;i++){

            char k=a.charAt(i);

            String aux=""+k;

            int x=Integer.parseInt(aux);

            s+=x;

        }

        System.out.println(s);

    }

}
```

3. Se dă un număr real. Se cere să se determine numărul de cifre de la partea întreagă.

Exemplu

Intrare	Ieșire
14.2019	2

Soluție:

```
import java.util.Scanner;

public class P1 {

    public static void main(String[] args) {

        Scanner cin=new Scanner(System.in);

        double a;

        a=cin.nextDouble();

        Double b=new Double(a);

        int y=(int)b.doubleValue();

        Stringaux=Integer.toString(y);

        System.out.println(aux.length());

    }

}
```

Probleme propuse

1.) a) Se cere să se creeze o clasă cu numele segment care să conțină datele și metodele:

- coordonatele capetelor unui segment: x1, y1, x2, y2
- constructor
- afișare
- lungimea segmentului

- b) În clasa `testsegment` definiți un obiect al clasei `segment` și verificați metodele.
- c) Citiți datele pentru n segmente și afișați segmentele ordonate după lungime.
- d) Afișați numărul de segmente de lungime minimă, respectiv maximă.

2. Creați o clasă cu numele funcției care să conțină:

- coeficienții unei funcții de gradul II: a, b, c
- x_1, x_2, x_v, y_v - rădăcinile și coordonatele vârfului $V(-b/(2a), -\Delta/(4a))$
- constructor
- rădăcini (determinați x_1, x_2)
- vârf (determinați x_v, y_v).

Se cere:

- a) Creați un obiect prin care să verificați metodele din clasă.
- b) Citiți n funcții de gradul 2, apoi afișați datele lor.
- c) Afișați datele funcțiilor cu vârful pe axa Ox , respectiv Oy .
- d) Afișați datele funcțiilor fără rădăcini reale.

3. Se dă un vector cu n componente numere naturale.

- a) Afișați numărul de numere pătrate perfecte din vector.
- b) Afișați pozițiile primului și ultimului număr pătrat perfect.

Exemplu

Intrare	Ieșire
8 20 3 100 7 144 25 1 200	a) 4 b) 3 7

4. Se dă un cuvânt, care se termină cu enter. Se cere să se afișeze literele egal depărtate de extreme care sunt diferite.

Exemplu

Intrare	Ieșire
maviap	m p v i

5. Se dă un cuvânt, care se termină cu enter. Verificați dacă există două caractere egale unul după altul.

Exemplu

Intrare	Ieșire
abcddb	DA

6. Se dă un text, care se termină cu enter, cuvintele sunt separate prin spații. Afișați cuvintele din text câte unul pe o linie.

Exemplu

Intrare	Ieșire
ana are mere	ana are mere

7. Se dau două cuvinte, unul sub altul, care se termină cu enter. Afișați literele comune o singură dată comune celor două cuvinte, separate prin câte un spațiu.

Exemplu

Intrare	Ieșire
Matematica timid	t i m

7. Moștenirea claselor

Noțiuni introductive

Prin *moștenire* (*extindere* sau *derivare*) înțelegem crearea unei clase folosind o clasă definită anterior.

Clasa pe care o moștenim se numește *superclasă*, iar clasa obținută prin moștenire se numește *subclasă*.

Observație

Subclasa preia de la *superclasă* datele și metodele fără a fi nevoie să le definim din nou. La acestea se pot adăuga noi date și metode, care trebuie definite în *subclasă*. Definirea *subclasei* presupune folosirea clauzei **extends** urmată de numele *superclasei*.

Exemplu

Definiți clasa **dreptunghi** cu datele membru: *a*, *b*, iar ca metode: constructor și afișare. Clasa **dreptunghi** o vom extinde la clasa **dreptunghicomplet** prin adăugarea metodelor: *diagonala*, *arie*, *perimetru*. Clasele necesare rezolvării acestei probleme sunt următoarele:

```
class dreptunghi{  
    int a, b;
```

```

void afis(){
    System.out.println(a + " " + b);
}

dreptunghi(int a, int b){
    this.a = a;
    this.b = b;
}
}

Class dreptunghicomplet extends dreptunghi{
    double diagonala(){
        return Math.sqrt(a*a + b*b);
    }
    int perimetru(){
        return 2*a + 2*b;
    }
    int arie(){
        return a*b;
    }
    dreptunghicomplet(int a, int b){
        super(a,b);
        //super se foloseste pentru a utiliza
        //constructorul din clasa pe care o extindem
    }
}

public class TestMostenire {
    public static void main(String[] args) {
        dreptunghi ol;
    }
}

```

```

        o1 = new dreptunghi(5,10);
dreptunghicomplet o2 = new dreptunghicomplet(10,20);
System.out.println("o1 : " );
        o1.afis();
System.out.println("o2 : ");
        o2.afis();
System.out.println("Diagonala : " + o2.diagonala());
System.out.println("Arie : " + o2.arie());
System.out.println("Perimetru : " + o2.perimetru());
    }
}

```

Observații

1) Dacă considerăm clasa C și C1 o altă clasă care moștenește clasa C, atunci se pot defini metode în C1 cu același nume ca în C. La fel și date membru. Implicit se folosesc cele din C1. Putem folosi date din C în obiecte ale clasei C1, prin prefixul **super**.

Exemplu

super.x - pentru date membru (x dată membru în C)

super(listă_parametri) - metodă constructor

super.numemetoda(listă_parametri) - pentru celelalte metode

2) Pentru constructorul din C1 putem folosi constructorul din C considerând ca primă instrucțiune din constructorul lui C1 apelul constructorului din C în formatul **super(listă_parametri)**.

Probleme rezolvate

1. a) Creați clasa fracție cu datele membru a, b pentru fracția a/b și metodele: constructor, afisare, simplificare. Extindeți clasa fracție la clasa operatiifracție astfel încât să adăugați metodele: adun, scad, impart, inmultesc pentru operațiile aritmetice adunare, scădere, împărțire și înmulțire. Verificați metodele pentru un obiect, apoi citiți datele pentru n fracții.

b) Afișați suma și produsul celor n fracții.

c) Afișați fracția cu numitorul cel mai mare din fracțiile date.

d) Afișați fracția cea mai mare din fracțiile date.

Soluție

```
import java.util.Scanner;

class fractie{

    int a, b;

    fractie(int a, int b){

        this.a = a;

        this.b = b;

    }

    void afisare(){

        System.out.println(a + " / " + b);

    }

    void simplificare(){

        int x = a, y = b;

        if(x*y != 0){

            while(x != y)

                if(x > y)

                    x -= y;
```

```

        else

            y -= x;

        a /= x;
        b /= x;
    }
}

}

class operatiifracție extends fracție{
    operatiifracție(int a, int b){
        super(a, b);
    }

    void adun(operatiifracție F){
        int x, y;

        x = a*F.b + b*F.a;

        y = b*F.b;

        a = x;

        b = y;

        simplificare();
    }

    void scad(operatiifracție F){
        int x, y;

        x = a*F.b - b*F.a;

        y = b*F.b;

        a = x;

        b = y;
    }
}

```

```

        simplificare();
    }

    void inmultesc(operatiifractie F){
        int x, y;

        x = a * F.a;
        y = b * F.b;

        a = x;
        b = y;

        simplificare();
    }

    void impart(operatiifractie F){
        int x, y;

        x = a * F.b;
        y = b * F.a;

        a = x;
        b = y;

        simplificare();
    }
}

public class Testfractie {
    public static void main(String[] args) {
        System.out.println("a) ");
        operatiifractie F,x;

        F = new operatiifractie(50, 70);
        F.afisare();
        F.simplificare();
    }
}

```

```

F.afisare();
System.out.println("Adunarea cu 10/30 :");
x = new operatiifractie(10, 30);
F.adun(x);
F.afisare();
System.out.println("Scaderea cu 10/30 :");
x = new operatiifractie(10, 30);
F.scad(x);
F.afisare();
System.out.println("Inmultirea cu 10/30 :");
x = new operatiifractie(10, 30);
F.inmultesc(x);
F.afisare();
System.out.println("Impartirea cu 10/30 :");
x = new operatiifractie(10, 30);
F.impart(x);
F.afisare();
operatiifractie f[];
Scanner cin=new Scanner(System.in);
int n = cin.nextInt();
f = new operatiifractie[n+1];
int a,b,i;
for(i=1;i<=n;i++){
    a=cin.nextInt();
    b=cin.nextInt();
    f[i]=new operatiifractie(a,b);
}

```

```

    }

    System.out.println("b) ");

    Operatiifractie s,p;

    s = new operatiifractie(0,1);
    p = new operatiifractie(1,1);
    for(i=1;i<=n;i++){
        s.adun(f[i]);
        p.inmultesc(f[i]);
    }

    System.out.print("Sumna= ");
    s.afisare();

    System.out.print("Produs= ");
    p.afisare();

    System.out.println("c) ");

    int Max=0;
    for(i=1;i<=n;i++)
        if(f[i].b>Max)
            Max=f[i].b;
    for(i=1;i<=n;i++)
        if(f[i].b == Max)
            f[i].afisare();

    System.out.println("d) ");

    double Max2=0,y;

    operatiifractie aux = null;
    for(i=1;i<=n;i++){
        y=f[i].a/f[i].b;

```



```

        if(y>Max2){
            Max2=y;
            aux=f[i];
        }
    }

    System.out.print("Fractia cea mai mare= ");
    aux.afisare();
}
}

```

2. Creați clasa **paralelipiped_dreptunghic** cu datele membru: a, b, c – lungimile muchiilor și metodele: constructor, afisare, arie_laterala, arie_totala și volum. Creați un obiect al acestei clase si verificați metodele.

Soluție

```

class paralelipiped_dreptunghic{
    int a, b, c;

    paralelipiped_dreptunghic(int a, int b, int c){
        this.a = a;
        this.b = b;
        this.c = c;
    }

    void afisare(){
        System.out.println(a + " " + b + " " + c);
    }

    int arie_laterala(){
        int P = 2*a + 2*b;
        return P*c;
    }
}

```

```

        int arie_totala(){
            return arie_lateralala() + 2*a*b;
        }

        int volum(){
            return a*b*c;
        }
    }

    public class Paralelipiped_dreptunghic_test {
        public static void main(String[] args){

            paralelipiped_dreptunghic x = new
            paralelipiped_dreptunghic(10,20,30);

            x.afisare();

            System.out.println("Arie laterala : " +
            x.arie_lateralala());

            System.out.println("Arie totala : " + x.arie_totala());

            System.out.println("Volum : " + x.volum());

        }
    }

```

3. Extindeți clasa de la problema 2 la clasa

paralelipiped_dreptunghic_extins

astfel încât să adăugați data membru x și metodele: constructor, modificare, afisare.

Metoda modificare crește lungimile muchiilor a, b, c cu x, iar afisare afișează a, b, c, x.

Creați un obiect și verificați metodele lui.

Soluție

```
class paralelipiped_dreptunghic_extins extends
paralelipiped_dreptunghic{

    int x;

    paralelipiped_dreptunghic_extins(int a, int b, int
c, int x){

        super(a, b, c);

        this.x = x;

    }

    void modificare(){

        a += x;

        b += x;

        c += x;

    }

    void afisare(){

        super.afisare();

        System.out.println("x = " + x);

    }

}
```

```
public class Paralelipiped_dreptunghic_test{

    public static void main(String[] args){

        paralelipiped_dreptunghic x = new
        paralelipiped_dreptunghic(10,20,30);

        x.afisare();

        System.out.println("Arie laterala : " +
        x.arie_laterala());

        System.out.println("Arie totala : " + x.arie_totala());

    }

}
```

```

System.out.println("Volum : " + x.volum());

paralelipiped_dreptunghic_extins y = new
paralelipiped_dreptunghic_extins(10,20,30,10);

y.modificare();

y.afisare();

System.out.println("Arie laterala : " +
y.arie_laterala());

System.out.println("Arie totala : " + y.arie_totala());

System.out.println("Volum : " + y.volum());
}

}

```

4.a) Se dau a și b numere naturale, $a < b$. Scrieți o clasă care să aibă ca date membru a și b și să conțină metodele: constructor, afișare, determinarea numărului de numere impare din intervalul $[a,b]$.

b) Extindeți clasa anterioară, astfel încât să definiți o nouă clasă care să conțină în plus data membru c și metodele: constructor, numărul de multipli ai lui c din $[a,b]$.

c) Definiți o a treia clasă care să conțină metoda `main` în care să se definească un obiect al subclasei definită la b) și să se apeleze toate metodele ei.

d) Citiți n obiecte ale clasei de la b) și afișați pentru fiecare numărul de numere impare și numărul de multipli ai lui c în intervalul $[a,b]$.

Soluție

```

import java.util.Scanner;

class interval{

    int a,b;

    interval(int a,int b){

        this.a=a;

        this.b=b;

    }
}

```

```

void afisare(){
    System.out.println("[ "+a+", "+b+" ]");
}

int nr_impere(){
    int i,nr=0;
    for(i=a;i<=b;i++)
        if(i%2==1)
            nr++;
    return nr;
}

}

class multiplii extends interval{
    int c;
    multiplii(inta,intb,int c){
        super(a,b);
        this.c=c;
    }
    intnr_multiplii(){
        int nr=0;
        for(int i=a;i<=b;i++)
            if(i%c==0)
                nr++;
        return nr;
    }
}

```

```

public class Problema4 {
public static void main(String[] args) {
Scanner cin=new Scanner(System.in);

    int a,b,c;

    System.out.print("a=");
    a=cin.nextInt();
    System.out.print("b=");
    b=cin.nextInt();
    System.out.print("c=");
    c=cin.nextInt();

    multiplii o=new multiplii(a,b,c);

    System.out.println("c");

    o.afisare();

    System.out.println("Numarul de numere impare: " +
o.nr_impare());

    System.out.println("Numarul de multipli ai lui " +
o.c + " : " + o.nr_multiplii());

    System.out.println("d");

    int n,i;

    n=cin.nextInt();

    multiplii x[]=new multiplii[n+1];

    for(i=1;i<=n;i++){

        a=cin.nextInt();

        b=cin.nextInt();

        c=cin.nextInt();

        x[i]=new multiplii(a,b,c);

    }
}

```

```

        for (i=1;i<=n;i++) {

            x[i].afisare() ;

            System.out.println(x[i].nr_impere() + " " +
            x[i].nr_multiplii());

        }

    }

}

```

5. a) Definiți o clasă cu numele **date_personale** care să conțină datele membru: nume, CNP, telefon, email și metodele afisare, construtor.

b) Moșteniți clasa **date_personale** și definiți clasa **elev** care să conțină în plus datele membru: scoala, clasa și metodele: constructor și afisare.

c) Creați un obiect al clasei elev și verificați metodele.

d) Citiți datele pentru n elevi și afișați elevii (datele membru) ordonați după clasă.

Soluție

```

import java.util.Scanner;

class date_personale{
    String nume,CNP,tel,email;

    date_personale(String nume, String CNP, String tel,
    String email){

        this.nume=nume;

        this.CNP=CNP;

        this.tel=tel;

        this.email=email;

    }

    void afisare(){

        System.out.println(nume+" "+CNP+" "+tel+" "+email);

    }

}

```

```

class elev extends date_personale{
    String scoala,clasa;
    elev(Stringa, String b, String c, String d, String
scoala, String clasa){
        super(a,b,c,d);
        this.scoala=scoala;
        this.clasa=clasa;
    }
    void afisare(){
        super.afisare();
        System.out.println(scoala+" "+clasa);
    }
}

public class Problema5 {
    public static void main(String[] args) {
        Scanner cin = new Scanner(System.in);
        String nume,CNP,tel,email,scoala,clasa;
        elev x;
        System.out.println("Date pentru un elev");
        nume=cin.nextLine();
        CNP=cin.nextLine();
        tel=cin.nextLine();
        email=cin.nextLine();
        scoala=cin.nextLine();
        clasa=cin.nextLine();
        x=new elev(nume,CNP,tel,email,scoala,clasa);
        x.afisare();
        int n;
        System.out.println("d");
    }
}

```



```

        n=cin.nextInt();
        cin.nextLine();
        elev e[]=new elev [n+1];
        for(int i=1;i<=n;i++){
System.out.println("Date pentru elevul "+i+" : ");
        nume=cin.nextLine();
        CNP=cin.nextLine();
        tel=cin.nextLine();
        email=cin.nextLine();
        scoala=cin.nextLine();
        clasa=cin.nextLine();
        e[i]=new elev (nume,CNP,tel,email,scoala,clasa);
    }
    elev aux;
    for(int i=1;i<n;i++)
        for(int j=i+1;j<=n;j++)
            if(e[i].clasa.compareTo(e[j].clasa)>0){
                aux=e[i];
                e[i]=e[j];
                e[j]=aux;
            }
    for(int i=1;i<=n;i++){
        System.out.println("elevul "+i+" : ");
        e[i].afisare();
    }
}
}

```

Probleme propuse

1. Creați clasa **patrat** cu data membru n (lungimea laturii pătratului) și ca metode: constructor și afisare.

a) Creați un obiect al acestei clase și verificați metodele ei.

b) Extindeți clasa **patrat** la clasa **patratextins** astfel încât să adăugați metodele: diagonala, perimetru, arie.

c) Creați un obiect al clasei **patratextins** și verificați metodele ei.

2. Folosind clasa **patratextins** de la problema 1 citiți datele pentru n pătrate și apoi afișați aceste pătrate ordonate crescător după lungimea laturii.

3. a) Se dau a și b numere naturale, $a < b$. Scrieți o clasă care să aibă ca date membru a și b și să conțină metodele: constructor, afișare, determinarea numărului de numere prime din intervalul $[a, b]$.

b) Extindeți clasa anterioară, astfel încât să definiți o nouă clasă care să conțină în plus datele membru c , d și metodele: constructor, numărul de multipli ai lui c , care nu sunt divizibili cu d din $[a, b]$.

c) Definiți o a treia clasă care să conțină metoda `main`, în care să se definească un obiect al subclasei definită la b) și să se apeleze toate metodele ei.

d) Citiți n obiecte ale clasei de la b) și afișați pentru fiecare numărul de numere prime și numărul de multipli ai lui c din $[a, b]$, care nu sunt divizibili cu d .

4. a) Definiți o clasă cu numele `date_referinta` care să conțină datele membru: nume, CI, telefon și metodele afisare, constructor.

b) Moșteniți clasa `date_referinta` și definiți clasa `colegi` care să conțină în plus datele membru: email, oras și metodele: constructor și afisare.

c) Creați un obiect al clasei `colegi` și verificați metodele ei.

d) Citiți datele pentru n colegi și afișați datele membru ordonate alfabetic după oraș.

8. Clase abstracte și Interfețe

Clase abstracte

Prin clasă *abstractă* înțelegem o clasă care are metode nedefinite (specificate doar prin antet în cadrul clasei). Aceste clase sunt precedate de cuvântul "abstract", la fel și metodele care nu sunt definite (numite *metode abstracte*).

Metodele nedefinite vor fi implementate în cadrul claselor care moștenesc clasele abstracte.

Probleme rezolvate

1. a) Definiți clasa abstractă **paralelipiped_dreptunghic_abstract** cu datele membru a, b, c și metodele neabstracte: constructor, afisare, respectiv arie_laterala, arie_totala, volum ca metode abstracte.

b) Extindeți clasa abstractă anterioară la clasa **paralelipiped**. Creați un obiect al clasei extinse și verificați metodele lui.

Pentru datele 10 20 30 avem:

Arie laterala : 1800

Arie totala : 2200

Volum : 6000

Soluție

```
abstract class paralelipiped_dreptunghic_abstract{  
    int a, b, c;
```

```

paralelipiped_dreptunghic_abstract(int a, int b, int c){
    this.a = a;
    this.b = b;
    this.c = c;
}

void afisare(){
    System.out.println(a + " " + b + " " + c);
}

abstract intarie_lateralala();
abstract intarie_totalala();
abstract int volum();
}

class paralelipiped extends
paralelipiped_dreptunghic_abstract{
    paralelipiped(int a, int b, int c){
        super(a, b, c);
    }

    int arie_lateralala(){
        int P = 2*a + 2*b;
        return P * c;
    }

    int arie_totalala(){
        return arie_lateralala() + 2*a*b;
    }
}

```

```

        int volum(){
            return a*b*c;
        }
    }

    public class Paralelipiped_test {
        public static void main(String[] args) {
            paralelipiped x = new paralelipiped(10, 20, 30);
            x.afisare();

            System.out.println("Arie laterala : " +
            x.arie_laterala());

            System.out.println("Arie totala : " + x.arie_totala());
            System.out.println("Volum : " + x.volum());
        }
    }

```

2. Creați n obiecte ale clasei paralelipiped și afișați aceste obiecte ordonate crescător după volum.

Exemplu

Pentru datele:

4

10 15 20

2 4 6

20 10 5

5 10 15

se va afișa:

2 4 6

5 10 15

20 10 5

10 15 20

Soluție

```
import java.util.Scanner;

abstract class paralelipiped_dreptunghic_abstract{
    int a, b, c;

    paralelipiped_dreptunghic_abstract(int a, int b, int c)
    {
        this.a = a;
        this.b = b;
        this.c = c;
    }

    void afisare(){
        System.out.println(a + " " + b + " " + c);
    }

    abstract intarie_laterala();
    abstract intarie_totala();
    abstract int volum();
}

class paralelipiped extends
    paralelipiped_dreptunghic_abstract{
    paralelipiped(int a, int b, int c){
        super(a, b, c);
    }
}
```

```

int arie_lateralala(){
    int P = 2*a + 2*b;
    return P * c;
}

int arie_totala(){
    return arie_lateralala() + 2*a*b;
}

int volum(){
    return a*b*c;
}
}

public class Paralelipiped_test2 {
    public static void main(String[] args) {
        Scanner cin = new Scanner(System.in);
        int n = cin.nextInt(), a, b, c, i, j;
        paralelipiped x[] = new paralelipiped[n+1];
        paralelipiped aux;
        for(i = 1; i <= n; i++){
            a = cin.nextInt();
            b = cin.nextInt();
            c = cin.nextInt();
            x[i] = new paralelipiped(a, b, c);
        }
    }
}

```

```

        for(i = 1; i < n; i++)
            for(j = i+1; j <= n; j++)
                if(x[i].volum() > x[j].volum()){
                    aux = x[i];
                    x[i] = x[j];
                    x[j] = aux;
                }
        System.out.println("=====");
        for(i = 1; i <= n; i++)
            x[i].afisare();
    }
}

```

Interfețe

Prin *interfață* înțelegem un proiect de clasă. O *interfață* conține doar constante și antete de metode. Acestea vor fi implementate ulterior. *Interfața* se definește folosind în loc de "class" cuvântul cheie "interface".

Problemă rezolvată

a) Creați interfața "cub" cu data membru: a - lungimea laturii și antetele metodelor: afisare, diagonala, volum.

b) Implementați interfața "cub" astfel încât să definiți clasa "cub_implementat". Apoi, creați un obiect al acestei clase și verificați metodele.

Exemplu

10

Volum : 1000

Diagonala : 17.32050807568877

Soluție

```
interface cub{
    int a = 10;
    public int volum();
    public double diagonala();
    public void afisare();
}

class cub_implementat implements cub{
    int a;
    cub_implementat(int a){
        this.a = a;
    }
    public int volum(){
        return a*a*a;
    }
    public void afisare(){
        System.out.println(a);
    }

    public double diagonala(){
        return a*Math.sqrt(3);
    }
}
```

```

public class Cub_test {
    public static void main(String[] args) {
        cub_implementat x = new cub_implementat(10);
        x.afisare();
        System.out.println("Volum : " + x.volum());
        System.out.println("Diagonala : " +
x.diagonala());
    }
}

```

Specificatori (modificatori)

Prin specificatori sau modificatori de acces înțelegem un atribut precizat la definirea clasei prin care se specifică modul de acces la date și metodele clasei.

Forma generală a unei clase este următoarea:

```

specificator class nume_clasa extends
nume_clasa_ce_se_mosteneste implements lista_interfete
{
    Date_membru
    Metode
}

```

Specificator poate fi:

- **public** => cu semnificația că această clasă poate fi accesată din orice program Java. Dacă acest specificator lipsește, atunci clasa poate fi accesată doar din pachetul unde aceasta se găsește.
- **abstract** => dacă lipsește clasa nu este abstractă și trebuie implementată complet.
- **final** => dacă se dorește ca această clasă să nu mai poată fi moștenită.

Datele membru și metodele pot avea și ele specificatori de acces.

Pentru date membru se pot folosi atributele:

- **public** => cu semnificația că data membru se poate accesa de oriunde și poate fi moștenită
- **protected** => cu semnificația că data membru se poate accesa doar în pachetul curent
- **private** => cu semnificația că data membru se poate accesa doar în clasa curentă și nu este moștenită
- **final** => cu semnificația că data membru este considerată constantă
- **static** => cu semnificația că data membru folosește aceeași memorie pentru toate obiectele

Pentru metode se folosesc aceleași atribute ca la datele membru, cu precizarea că **final** nu permite ca metoda să fie redefinită prin moștenire, iar **static** faptul că metoda nu este reținută de fiecare obiect.

Probleme propuse

1. Creați clasa abstractă **patrat** cu data membru n (lungimea laturii pătratului) și ca metode neabstracte: constructor și afisare, iar ca metode abstracte diagonala, perimetru, arie.

a) Creați un obiect al acestei clase și verificați metodele ei.

b) Extindeți clasa **patrat** la clasa **patratextins** astfel încât să definiți metodele: diagonala, perimetru, arie.

c) Creați un obiect al clasei **patratextins** și verificați metodele ei.

2. Folosind clasa **patratextins** de la problema 1 citiți datele pentru n pătrate și apoi afișați aceste pătrate ordonate descrescător după arie, iar la arii egale după perimetru.

3. a) Se dau a și b numere naturale, $a < b$. Scrieți o interfață care să aibă ca date membru a și b și să conțină metodele: constructor, afișare, determinarea numărului de numere prime din intervalul $[a, b]$.

b) Implementați interfața de la cerința a) pentru a obține o nouă clasă.

c) Definiți o clasă care să conțină metoda `main`, în care să se utilizeze un obiect al clasei definită la subpunctul b) și să se apeleze toate metodele ei.

4. Creați o clasă cu numerele **cerc** care să conțină datele membru: x , y , r și metodele neabstracte: constructor, afisare (pentru afișarea datelor membru), iar ca metode abstracte: `arie`, `lungime`, constructor (pentru calculul ariei și lungimii cercului, respectiv pentru inițializarea datelor membru)

Extindeți clasa **cerc** astfel încât să definiți metodele abstracte:

- `arie`

- `lungime`

- constructor

Noua clasă se va numi **cercOp**. Folosiți clasa **cercOp** pentru a defini un obiect cu x , y , r - citite de la tastatură și apoi apelați metodele obiectului.

9. Excepții. Clasa StringTokenizer. Fișiere text

Excepții

Prin *excepție* vom înțelege o situație nedorită în care poate ajunge un program în timpul rulării. Iată câteva exemple de astfel de situații:

- Vrem să citim un număr întreg și introducem un caracter care nu este cifră.
- Am declarat un vector x cu 100 componente (indicii sunt în intervalul $0..99$) și utilizăm $x[101]$.
- Deschidem un fișier text pentru citire și el nu există.
- Efectuăm o împărțire la 0.

Având în vedere aceste aspecte, ne punem întrebarea: cum putem să nu oprim din rulare un program care trece prin situații de excepție?

Tratarea acestor erori se poate face utilizând clasa **Exception**.

Exemplu

Citirea unui număr n de două cifre ($9 < n < 100$).

```
import java.util.Scanner;

class eroare extends Exception{

    void afis(){

        System.out.println("numarul nu are douacifre");

    }

}
```

```

class citire{
public static int cit() throws eroare{
    int n;

    Scanner cin = new Scanner(System.in);

    n = cin.nextInt();

    if(n > 9 && n < 100)

        return n;

    throw new eroare();
}

public static void main(String[] args){
    try {
        int nr;

        nr = cit();

        System.out.println("Am citit n = " + nr);
    }

    catch (eroare er){
        er.afis();
    }
}
}

```

Observații

1. Clauza **throws** (*aruncă* în limba română) are rolul de a specifica faptul că o metodă poate întoarce și referințe către obiectele altor clase.
2. Instrucțiunea **throw** are ca efect returnarea unei referințe către unul din obiectele clasei specificate după clauza **throws** (clasele sunt separate prin caracterul virgulă, dacă sunt mai multe).

3. Forma generală a instrucțiunii **try** este următoarea:

```
try{

    //secventa de instructiuni ce poate conduce la
    exceptii

}

catch(eroare1 er1){

    //secventa de instructiuni ce trateaza exceptia
    folosind clasa eroare1

}

catch(eroare2 er2){

    //secventa de instructiuni ce trateaza exceptia
    folosind clasa eroare2

}

...
```

Opțional la sfârșitul acestei instrucțiuni poate fi utilizată clauza **finally** cu o secvență de instrucțiuni ce se execută indiferent dacă a fost eroare sau nu.

4. În exemplu utilizarea metodei `cit` se face în mod obligatoriu, cu ajutorul instrucțiunii **try** (*încearcă* în limba română). Dacă citirea reușește, programul își încheie execuția, altfel, se verifică dacă obiectul întors de metodă este de tip “eroare”. În caz afirmativ se execută metoda **afis** a acestui obiect.

StringTokenizer

Clasa **StringTokenizer** ne permite să prelucrăm cu ușurință entități (cuvinte, secvențe de caractere) dintr-un șir de caractere separate prin caractere specificate.

Clasa **StringTokenizer** conține următorii constructori:

- **StringTokenizer(String s)** => data membru din obiectul curent este șirul de caractere din `s`

- **StringTokenizer(String s, String t)** => data membru din obiectul curent este șirul de caractere din s, iar separatori în s sunt considerați caracterele lui t.

Clasa **StringTokenizer** ne permite sa extragem ușor cuvintele din s separate de caractere din t, dacă există t ca parametru. Altfel, separatorii sunt ' ', '\n', '\t'(tab).

Alte metode utile:

- **nextToken()** -> returnează următoarea entitate din obiect;
- **hasMoreTokens()** -> returnează true dacă mai există cuvinte de parcurs și false altfel;
- **countTokens()** -> returnează numărul de cuvinte din obiect;

Observație

Clasa StringTokenizer se găsește în pachetul java.util.

Exemplu

Se dă un șir de caractere. Afișați cuvintele formate din 3 litere aflate înșirul de caractere.

Pentru șirul de caractere "Vasile are trei mere si doi prieteni" se va afișa:

are

doi

Soluție

```
import java.util.*;
import java.util.Scanner;

public class ExempluStringtokenizer{

    public static void main(String[] args){
```



```

Scanner cin = new Scanner(System.in);

String s = cin.nextLine();

StringTokenizer t = new StringTokenizer(s, ",. ;");

while(t.hasMoreTokens()){

    s = t.nextToken();

    if(s.length() == 3)

        System.out.println(s);

}

}

}

```

Fişiere text

Pentru citirea si afişarea datelor din fişier folosim clase care se găsesc în pachetul java.io.

Citirea presupune utilizarea a trei obiecte din clasele:

- **FileInputStream**
- **InputStreamReader**
- **BufferedReader**

Afişarea presupune utilizarea a două obiecte din clasele:

- **FileOutputStream**
- **PrintStream**

Probleme rezolvate

1. Se dă n număr natural nenul mai mic sau egal cu 10000. Afișați cu câte un spațiu între ele numerele pătrate perfecte mai mici sau egale cu n .

Exemplu

Date.in	Ieșire
24	0 1 4 9 16

Soluție

```
import java.io.*;
import java.util.*;

public class Fisier1 {

    public static void main(String[] args) throws
        IOException{

        FileInputStream f = new FileInputStream("date.in");
        InputStreamReader fchar = new InputStreamReader(f);
        BufferedReader buf = new BufferedReader(fchar);

        int n, i;

        n = Integer.parseInt(buf.readLine());

        fchar.close();

        FileOutputStream g = new FileOutputStream("date.out");
        PrintStream gchar = new PrintStream(g);

        for(i = 0 ; i <= Math.sqrt(n); i++)

            gchar.print(i * i + " ");

        gchar.close();

    }

}
```

Observație

Dacă fișierele de intrare și ieșire se găsesc în alte foldere, decât cel curent, atunci trebuie precizată calea acestuia în obiectele create.

Exemplu

C:\\Users\\Student\\Documents\\NetBeansProjects\\fisier1\\src\\fisier1\\date.out

2. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați vectorul ordonat crescător.

Exemplu

date.in 5 7 2 9 3 8	date.out 2 3 7 8 9
----------------------------------	------------------------------

Soluție

```
import java.io.*;

import java.util.*;

public class Fisiere2 {

    public static void main(String[] args) throws
        IOException{

        FileInputStream f = new FileInputStream("date.in");
        InputStreamReader fchar = new InputStreamReader(f);
        BufferedReader buf = new BufferedReader(fchar);

        int n, x[], i, j, aux;

        String s;

        n = Integer.parseInt(buf.readLine());

        x = new int[n+1];

        s = buf.readLine();

        StringTokenizer t = new StringTokenizer(s);
```

```

for(i = 1; i <= n; i++)
    x[i] = Integer.parseInt(t.nextToken());
FileOutputStream g = new FileOutputStream("date.out");
PrintStream gchar = new PrintStream(g);
for(i = 1; i < n; i++)
    for(j = i+1; j <= n; j++)
        if(x[i] > x[j]){
            aux = x[i];
            x[i] = x[j];
            x[j] = aux;
        }
for(i = 1; i <= n; i++)
    gchar.print(x[i] + " ");
}
}

```

3. Se dă un tablou cu m linii și n coloane, componentele sunt numere naturale. Afișați liniile tabloului care încep și se termină cu un număr par.

Exemplu

date.in	date.out
4 5	8 3 1 2 4
1 2 3 4 5	4 2 6 5 2
8 3 1 2 4	
3 5 6 9 8	
4 2 6 5 2	

Soluție

```

import java.util.*;
import java.io.*;

```

```

public class Fisiere3 {
    public static void main(String[] args) throws
        IOException{
        FileInputStream f = new FileInputStream("date.in");
        InputStreamReader fchar = new InputStreamReader(f);
        BufferedReader buf = new BufferedReader(fchar);
        int m, n, a[][] , i, j;
        String s;
        s = buf.readLine();
        StringTokenizer t = new StringTokenizer(s);
        m = Integer.parseInt(t.nextToken());
        n = Integer.parseInt(t.nextToken());
        a = newint[m+1][n+1];
        for(i = 1; i <= m; i++){
            s = buf.readLine();
            t = new StringTokenizer(s);
            for(int j = 1; j <= n; j++)
                a[i][j] = Integer.parseInt(t.nextToken());
        }
        FileOutputStream g = new FileOutputStream("date.out");
        PrintStream gchar = new PrintStream(g);
        for(int i = 1; i <= m; i++)
            if(a[i][1] % 2 == 0 && a[i][n] % 2 == 0){
                for(int j = 1; j <= n; j++)
                    gchar.print(a[i][j] + " ");
                gchar.println();
            }
        fchar.close();
        gchar.close();
    }
}

```

Probleme propuse

1. Se dau două numere naturale a, b , cu $a < b \leq 100000$. Afișați numerele palindrom din $[a, b]$.

Exemplu

date.in 452 480	date.out 454 464 474
---------------------------	--------------------------------

2. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați componentele vectorului care au ultima cifră 3 sau 4.

Exemplu

date.in 4 80 53 524 78	date.out 53 524
-------------------------------------	---------------------------

3. Se dau doi vectori x, y cu m , respectiv n cu componente numere naturale distincte de tip `int`. Se cere să se afișeze componentele din x care nu sunt în y .

Exemplu

date.in 4 3 6 4 8 5 7 8 9 1 6	date.in 3 4
--	-----------------------

4. Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați perechile de componente din vector cu suma un număr prim, pe rânduri diferite.

Exemplu

date.in 5 4 5 3 8 30	date.out 4 3 5 8 3 8
-----------------------------------	--------------------------------------

5.Se dă un vector cu n componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați cea mai mare sumă divizibilă cu n de componente aflate pe poziții consecutive elemente din vector.

Exemplu

date.in 4 8 2 21 3	date.out 24 (21, 3)
---------------------------------	-------------------------------

6. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m, n \leq 100$. Afișați pe rânduri diferite liniile formate numai din numere care sunt cifre.

Exemplu

Intrare 4 5 1 2 7 3 4 7 5 9 61 200 4 8 5 8 3 3 9 45 100 109	Ieșire 1 2 7 3 4 4 8 5 8 3
---	---

7. Se dă un tablou pătratic de dimensiune n cu componente numere naturale mai mici sau egale cu 10000, $1 \leq n \leq 100$. Afișați pentru fiecare diagonală numele neprime, pe câte o linie separate prin câte un spațiu (diagonala principală, respectiv secundară).

Exemplu

date.in 4 1 2 7 3 7 5 9 6 4 8 7 8 30 9 45 10	date.in 1 10 9 8 30
--	----------------------------------

8. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m, n \leq 100$. Afișați pe rânduri diferite coloanele ordonate crescător.

Exemplu

date.in	date.out
4 5	2 5 8 9
1 2 7 3 4	3 61 80 100
7 5 9 61 200	
4 8 5 80 3	
3 9 45 100 109	

9. Se dă un tablou bidimensional cu m linii, n coloane numere naturale mai mici sau egale cu 10000, $1 \leq m$, $n \leq 100$. Afișați pe rânduri diferite liniile cu numărul de numere nule maxim.

Exemplu

Intrare	Ieșire
4 5	1 0 0 0 1
1 0 0 0 1	0 5 0 1 0
2 0 1 1 1	
0 5 0 1 0	
1 1 1 1 1	

10. Noțiuni introductive de programare vizuală

Crearea ferestrelor

Clasa **JFrame** din pachetul **javax.swing.*** permite crearea obiectelor ce conțin ferestre.

Metode specifice:

- **JFrame()** - constructor pentru fereastră fără nume
- **JFrame(String titlu)** - constructor pentru fereastră cu nume: nume
- **void setSize(int latime, int inaltime)** - pentru configurarea dimensiunilor ferestrei
- **void setLocation(int x0, int y0)** - pentru poziționarea ferestrei pe desktop, colțul din stânga sus va fi de coordonate (**x0**, **y0**) în sistemul de coordonate al ecranului.
- **void setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)** - închide programul odată cu fereastra
- **void setResizable(boolean val)** - pentru a permite sau nu modificarea dimensiunilor ferestrei
- **setVisible(boolean val)** - stabilește dacă fereastra este vizibilă sau nu

Exemplu

Creați o fereastră ca cea din figura 10.1.

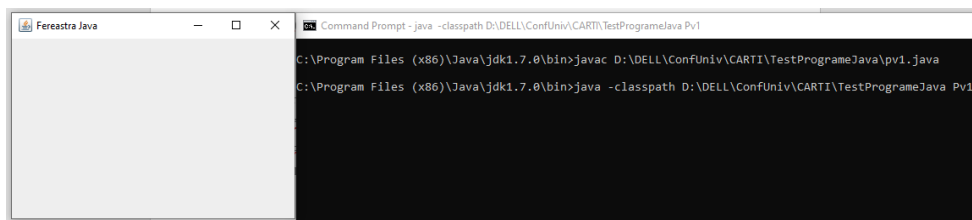


Figura 10.1 Crearea unei ferestre folosind un program cu numele Pv1.java

```
import javax.swing.*;

public class Pv1 {

    public static void main(String[] args) {

        JFrame fer = new JFrame("Fereastră Java");

        fer.setSize(350,250);

        fer.setLocation(100,200);

        fer.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        fer.setVisible(true);

    }

}
```

Observație

Este indicat ca **JFrame** să se moștească, pentru a obține o clasă cu constructor ce configurează fereastra rezultată.

Programul anterior se poate rescrie astfel:

```
class Fer extends JFrame{

    Fer(String nume, int l, int L, int x, int y){

        super(nume);

        setSize(l,L);

        setLocation(x,y);

    }

}
```

```

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setVisible(true);
    }
}

public class Pv2 {

    public static void main(String[] args) {

        Fer f = new Fer("Fereastra Java", 350, 250, 100, 200);

    }
}

```

Containere

În general ferestrele conțin o gamă variată de obiecte (butoane, meniuri, texte, imagini, etc.). Structura care reține referințele către obiectele ce se află pe fereastră este un obiect al clasei **Container**. Accesul la container-ul unei ferestre se face utilizând o metoda din clasa **JFrame**, cu numele **getContentPane()**, care returnează o referință către containerul ferestrei. Pornind de la această referință putem adăuga componente și specifica modul lor de aranjare într-o fereastră.

1. Adăugarea componentele (obiectelor) dorite în fereastră, folosind metoda:

Component add(Component componenta) - adaugă o componentă ferestrei

2. Precizarea modului de aranjare în fereastră a componentelor, folosind gestionari de poziționare (ce aranjează automat componentele unui container)

Pentru a atașa unui container un gestionar de poziționare se folosește metoda:

void setLayout(LayoutManager gestionar)

Exemplu de gestionar: **FlowLayout** –aranjează componentele în fereastră pe linii – unele după altele.

Observații

- Clasa Container se găsește în pachetul `java.awt`.
- Butoanele sunt component de tip `JButton`, care au un constructor de tip `JButton(String s)`. Șirul de caractere `s` va apărea pe suprafața butonului.

Exemplu

Creați o fereastră care să conțină trei butoane ca în figura 10.2.

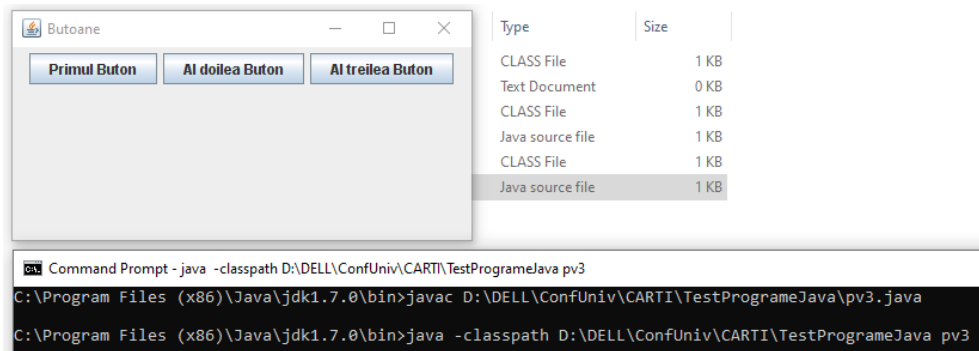


Figura 10.2 Fereastră cu 3 butoane

```
import javax.swing.*;
import java.awt.*; //pentru clasa JButton si Container
class fereastra extends JFrame{
fereastra(String s, int L, int l, int x, int y){
    super(s);
    setSize(L, l);
    setLocation(x, y);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
    //crearea containerului
    Container C = getContentPane();
```

```

        //modul de aranjare al obiectelor in container
        C.setLayout(newFlowLayout());

        JButton B1 = new JButton("Primul Buton");
        C.add(B1);

        JButton B2 = new JButton("Al doilea Buton");
        C.add(B2);

        JButton B3 = new JButton("Al treilea Buton");
        C.add(B3);

    }

}

public class pv3 {

    public static void main(String[] args) {

        fereastră f = new fereastră("Butoane", 400,
200, 0, 0);

    }

}

```

Evenimente

În Java există o interfață numită **ActionListener**, “ascultător” de evenimente de tip **ActionEvent**. Un exemplu de eveniment de tipul **ActionEvent** este “apăsarea” unui buton. Interfața conține antetul unei singure metode:

```
actionPerformed(ActionEvent e)
```

Pentru ca o componentă să poată răspunde la un eveniment de tipul **ActionEvent**, trebuie să se implementeze clasa **ActionListener**. Adică, clasa care include componenta (fereastră) trebuie să conțină clauza:

```
implements ActionListener;
```

și să implementeze metoda **actionPerformed**. Această metodă se va executa automat atunci când este apăsat butonul. **ActionEvent** este o clasa care conține metoda:

String getActionCommand() - care returnează șirul de caractere ce a transmis evenimentul.

Observație

Pentru a programa mai eficient este de preferat să se scrie câte o metodă corespunzătoare fiecărui buton. Metodele acestea se vor apela în metoda **actionPerformed**.

Exemplu

Modificați programul din exemplul anterior, astfel încât să se afișeze șirul de caractere de pe butonul apăsat (figura 10.3).

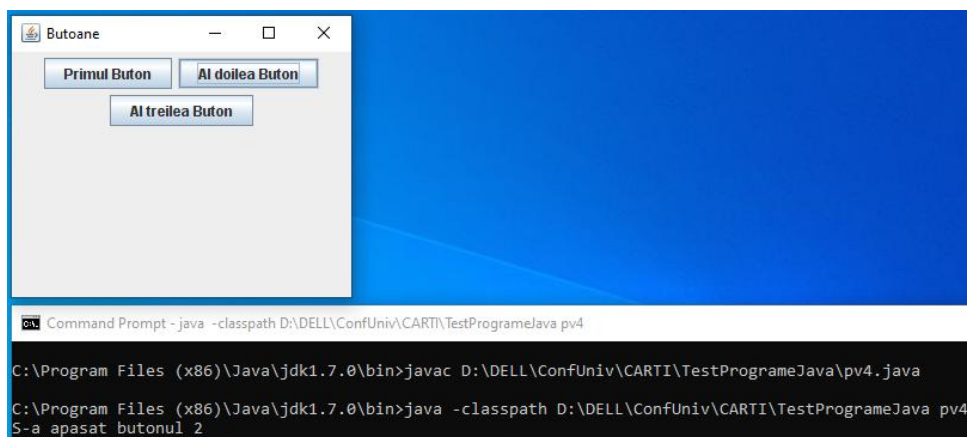


Figura 10.3 Aplicație cu evenimente

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*; //pentru clasa JButton si Container
class fereastră extends JFrame implements
ActionListener{
fereastră(String s, int L, int l, int x, int y){
```

```

super(s);
setSize(L, 1);
setLocation(x, y);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setVisible(true);
//crearea containerului
Container C = getContentPane();
//modul de aranjare al obiectelor in container
C.setLayout(new FlowLayout());
JButton B1 = new JButton("Primul Buton");
B1.addActionListener(this);
C.add(B1);
JButton B2 = new JButton("Al doilea Buton");
B2.addActionListener(this);
C.add(B2);
JButton B3 = new JButton("Al treilea Buton");
B3.addActionListener(this);
C.add(B3);
}

public void actionPerformed(ActionEvent e){
if(e.getActionCommand().compareTo("Primul Buton") == 0)
    actiune1();
if(e.getActionCommand().compareTo("Al doilea Buton") == 0)
    actiune2();
if(e.getActionCommand().compareTo("Al treilea Buton") == 0)
    actiune3();
}

```

```

void actiune1() {
    System.out.println("S-a apasat butonul 1");
}

void actiune2() {
    System.out.println("S-a apasat butonul 2");
}

void actiune3() {
    System.out.println("S-a apasat butonul 3");
}
}

public class pv4 {
    public static void main(String[] args) {
        fereastră f = new fereastră("Butoane", 300, 250, 0, 0);
    }
}

```

Ferestre pentru afișarea datelor de ieșire

- Metoda **JOptionPane.showConfirmDialog** afișează șirul de caractere specificat ca parametru într-o fereastră de dialog cu butoane de confirmare.

```
JOptionPane.showConfirmDialog(this, "Sir de caractere")
```

- Metoda **ShowMessageDialog** din clasa **JOptionPane** afișează șirul de caractere specificat ca parametru într-o fereastră cu un singur buton.

```
JOptionPane.showMessageDialog(this, "Sir de  
caractere");
```


Exemplu

Pentru problema anterioară putem face afișarea în modul următor:

```
void actiune1() {  
    JOptionPane.showMessageDialog(this, "S-a apasat butonul  
1");  
}  
  
void actiune2() {  
    JOptionPane.showMessageDialog(this, "S-a apasat butonul  
2");  
}  
  
void actiune3() {  
    JOptionPane.showMessageDialog(this, "S-a apasat butonul  
3");  
}
```

Probleme propuse

1. Scrieți un program care să creeze o fereastră cu titlul numele orașului vostru, poziționată în dreapta-sus.
2. Scrieți un program care să afișeze o fereastră cu două butoane. La apăsarea primului buton se vor afișa numerele pare de două cifre, iar la apăsarea celui de-al doilea buton numerele impare de două cifre.
3. Scrieți un program care să afișeze o fereastră cu două butoane. La activarea primului buton se citește n , iar la activarea celui de-al 2-lea buton se vor afișa numerele $n-1$, n , $n+1$.
4. Creați o aplicație vizuală cu trei butoane: unul pentru citirea unui vector, al doilea pentru afișarea minimului din vector, iar al treilea pentru afișarea maximului din vector.
5. Se dau n , k numere naturale nenule cu cel mult 10 cifre fiecare în fișierul cu numele date.in. Verificați dacă n se poate scrie ca sumă de k numere naturale distincte. În caz afirmativ se va afișa o soluție. Scrieți o aplicație vizuală cu două butoane, unul pentru citirea datelor de intrare, iar celălalt pentru afișarea datelor de ieșire.

11. Componente folosite în programarea vizuală

11.1 Componente de tip JButton

Un obiect al clasei JButton este un buton. Constructorii folosiți pentru această clasă sunt:

- **JButton()** –pentru un buton fără text și fără imagine
- **JButton(String text)**- pentru un buton pe care este scris șirul de caractere *text*
- **JButton(Icon icon)**- pentru un buton format cu o imagine de mici dimensiuni

Exemplu

```
Icon icon=new ImageIcon("p.jpg");
```

```
JButton A=new JButton(icon);
```

- **JButton(String text, Icon icon)**- pentru un buton pe care este scris cu text peste o imagine

Exemplu

```
Icon icon = new ImageIcon("p.jpg");
```

```
JButton A = new JButton("Java", icon);
```

Pentru poziționarea textului pe buton se mai folosesc metodele:

- **setVerticalTextPosition(*constanta*)** - setează poziția pe verticală a textului în raport cu imaginea. *constanta* poate fi **JButton.TOP**, **JButton.BOTTOM**, **JButton.CENTER**, pentru deasupra, dedesuptul, respectiv centrul imaginii.

- **setHorizontalTextPosition(*constanta*)** - setează poziția pe orizontală a textului în raport cu imaginea. *constanta* poate fi **JButton.LEFT**, **JButton.RIGHT**, **JButton.CENTER**, pentru stânga, dreapta, respectiv centrul imaginii.

Observație

Mărimea imaginii determină mărimea butonului.

11.2. Componente de tip JLabel

Obiectele clasei **JLabel** permit afișarea de texte și imagini. Clasa **JLabel** conține câteva metode necesare pentru utilizarea obiectelor:

- **JLabel(String s)** - afișează șirul de caractere din s

Exemplu

```
Container x = getContentPane();  
x.setLayout(new BorderLayout());  
JLabel A = new JLabel("Un text...");  
x.add(A);
```

- **JLabel(Icon icon)** - afișează o imagine

Exemplu

```
Container x = getContentPane();  
x.setLayout(new BorderLayout());  
Icon icon = new ImageIcon("p.jpg");  
JLabel A = new JLabel(icon);  
x.add(A);
```

- **JLabel(String text, Icon icon, int aliniereorizontala)**- afișează o imagine și un text. Ansamblul imagine+ text se aliniază pe orizontală, conform cu ultimul parametru (**CENTER**, **RIGHT** sau **LEFT**)

- **setVerticalTextPosition(constanta)**

- **setHorizontalTextPosition(constanta)**

constanta are aceleași valori ca la **JButton**.

11.3. Componente de tip JPanel

Componentele **JPanel** sunt de tip container, adică au rolul de a conține pe suprafața lor alte componente. Clasa conține câteva metode utilizate frecvent:

- **JPanel()**- constructor, obiectul rezultat are gestionarul de poziționare **FlowLayout()**;

- **JPanel(LayoutManager gestionar)**- constructor cu precizarea gestionarului de obiecte (**GridLayout()**, **BorderLayout()**, etc.)

- **add(Component c)** - adaugă componenta c containerului.

11.4 Componenta JTextField

Componentele de tip **JTextField** - sunt folosite pentru ca utilizatorul să introducă sau să afișeze șiruri de caractere de la tastatură într-o fereastră.

Metodele cele mai folosite din clasa **JTextField** sunt:

- **JTextField(int nr)**- creează un edit vid, care are o lățime suficientă pentru a vizualiza **nr** caractere. Șirul introdus poate avea oricâte caractere.

- **JTextField(String s)**- creează un edit care inițial afișează un șir de caractere(din obiectul **s**)

- **JTextField(String s, int nr)** - creează un edit de dimensiune **nr** caractere, care inițial afișează un șir de caractere (din obiectul **s**)

- **String getText()**- returnează șirul de caractere reținut de edit la un moment dat.

Exemplu

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener{
    JTextField text;

    public Fer(String titlu){
        super(titlu);
        setSize(300,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x = getContentPane();
        x.setLayout(new FlowLayout());
        JLabel et = new JLabel("Orasul:");
        x.add(et);
        text = new JTextField(15);
        x.add(text);
        JButton but = new JButton("Ce s-a introdus?");
        x.add(but);
        but.addActionListener(this);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e){
        String txt = e.getActionCommand();
        if(txt.compareTo("Ce s-a introdus?")==0)
            System.out.println(text.getText());
    }
}
```

```

@Override // suprascrierea metodei
public void actionPerformed(ActionEvent e) {
    throw new UnsupportedOperationException("Not supported
yet.");
}

}

public class Pv2 {
    public static void main(String[] args) {
        Fer f=new Fer("Exemplu de edit");
    }
}

```

Problemă rezolvată

Creați o aplicație vizuală care să citească un număr natural n și să afișeze: pătratul, radicalul și divizorii lui n .

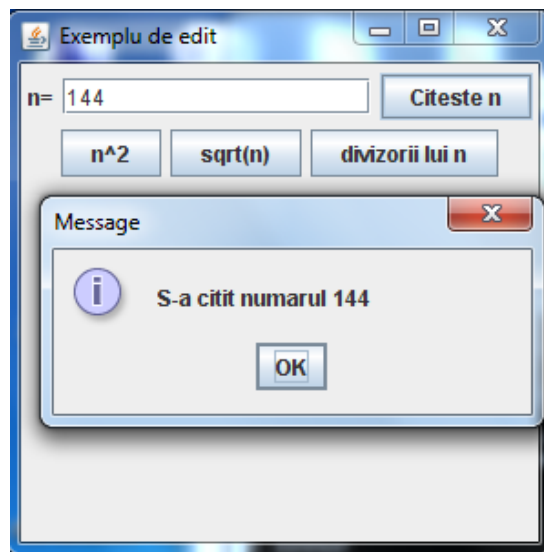


Figura 11.1: Exemplu de utilizare a aplicației

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Fer extends JFrame implements ActionListener {
    JTextField txt;
    int n;

    public Fer(String titlu) {
        super(titlu);
        setSize(300, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Container x = getContentPane();
        x.setLayout(new FlowLayout());
        JLabel et = new JLabel("n=");
        x.add(et);
        txt = new JTextField(15);
        x.add(txt);
        JButton A = new JButton("Citeste n");
        x.add(A); A.addActionListener(this);
        JButton B = new JButton("n^2");
        x.add(B); B.addActionListener(this);
        JButton C = new JButton("sqrt(n)");
        x.add(C); C.addActionListener(this);
        JButton D = new JButton("divizorii lui n");
        x.add(D); D.addActionListener(this);
        setVisible(true);
    }
}

```

```

public void actionPerformed(ActionEvent e){
    if(e.getActionCommand().compareTo("Citeste n")==0){
        n=Integer.parseInt(txt.getText());
        JOptionPane.showMessageDialog(this,"S-a citit numarul
        "+n);
    }
    if(e.getActionCommand().compareTo("n^2")==0){
        JOptionPane.showMessageDialog(this,"Patratul lui "+n+"
        este "+n*n);
    }
    if(e.getActionCommand().compareTo("sqrt(n)")==0){
        JOptionPane.showMessageDialog(this,"Radical din "+n+"
        este "+Math.sqrt(n));
    }
    if(e.getActionCommand().compareTo("divizorii lui
    n")==0){
        int k;
        String s;
        s="";
        for(k=1;k<=n;k++)
            if(n%k==0)
                s+=k+" ";
        JOptionPane.showMessageDialog(this,"Divizori "+s);
    }
}

public class pvtext{
    public static void main(String s[]){
        Fer fp=new Fer("Exemplu de edit");
    }
}

```


Probleme propuse

1. Scrieți un program care să creeze o fereastră ce să conțină textul “Introduceți un cuvânt: ”, un edit în care să se poată introduce un cuvânt și un buton, care să permită activarea verificării condiției de cuvânt palindrom (citit de la stânga la dreapta sau invers se obține același cuvânt, exemplu: **radar**).
2. Scrieți un program care să afișeze o fereastră cu două edituri. La apăsarea unui buton se va activa verificarea condiției de șiruri de caractere identice (introduse în edituri).
3. Scrieți un program care să aibă interfața ca în problema rezolvată, doar că cele trei cerințe vor fi: afișarea inversului lui n , afișarea cifrei maxime din n , respectiv afișarea numărului maxim ce se poate forma cu cifrele lui n .
4. Creați o aplicație vizuală, care permite citirea a trei numere și verificarea condiției de lungimi ale laturilor unui triunghi.
5. Creați o aplicație vizuală care citește coordonatele a trei puncte în plan și afișează aria și perimetrul triunghiului cu vârfurile în aceste puncte.

12. Fire de execuție

12.1 Noțiuni introductive

“Multithreading” reprezintă capacitatea unui program de a executa mai multe secvențe de instrucțiuni în același timp. O astfel de secvență de instrucțiuni se numește *fir de execuție* sau *thread*. Limbajul Java suportă multithreading prin clase disponibile în pachetul `java.lang`. În acest pachet există 2 clase **Thread** și **ThreadGroup** și interfața **Runnable**. Clasa **Thread** și interfața **Runnable** oferă suport pentru lucrul cu **thread**-uri ca entități separate, iar clasa **ThreadGroup** pentru crearea unor grupuri de **thread**-uri în vederea tratării acestora într-un mod unitar.

Există 2 metode pentru crearea unui fir de execuție: se creează o clasă derivată din clasa **Thread** sau se creează o clasă care implementează interfața **Runnable**.

12.2 Crearea unui fir de execuție prin extinderea clasei Thread

Pentru crearea firelor de execuție folosind clasa **Thread** se parcurg etapele:

- se creează o clasă ce moștenește clasa **Thread**
- se suprascrie metoda **run()** moștenită din clasa **Thread**
- se instanțiază un obiect **thread** folosind **new**
- se pornește thread-ul instanțiat, prin apelul metodei **start()** moștenită din clasa **Thread**. Apelul acestei metode face ca mașina virtuală Java să creeze contextul de program necesar unui **thread**, după care să apeleze metoda **run()**.

Exemplu

```
public class Fir {  
    public static void main(String args[]) {  
        FirDeExecutie fir = new FirDeExecutie();  
        fir.start();  
        System.out.println("Ne intoarcem la main");  
    }  
}  
  
class FirDeExecutie extends Thread {  
    public void run() {  
        int i;  
        for (i = 0; i < 100; i++)  
            System.out.println("Pasul " + i);  
        System.out.println("functia run s-a  
terminat");  
    }  
}
```

Observație

Metoda **main** are propriul fir de execuție. Prin apelul metodei **start** se cere mașinii virtuale Java crearea și pornirea unui nou fir de execuție. Din funcția **start** se va ieși imediat. Firul de execuție corespunzător metodei **main** își va continua execuția independent de noul fir de execuție creat.

Să considerăm un alt exemplu simplu în care se vor crea două fire de execuție ce rulează în același timp. Metoda **sleep** cere oprirea rulării firului de execuție curent pentru un interval de timp specificat.

```

public class Fir1 {
    public static void main(String args[]) {
        FirDeExecutie fir1 = new FirDeExecutie();
        FirDeExecutie fir2 = new FirDeExecutie();
        fir1.start();
        fir2.start();
        System.out.println("Revenim la main");
    }
}

class FirDeExecutie extends Thread {
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("Pasul " + i);
            try {
                sleep(500);
                //oprirea pt. 0,5 secunde
                //a firului de executie
            } catch (InterruptedException e) {
                System.err.println("Eroare");
            }
        }
        System.out.println("Run s-a terminat");
    }
}

```

12.3 Prioritățile firelor de execuție

Limbajul Java definește 3 constante pentru stabilirea priorităților firelor de execuție:

```
public final static int MAX_PRIORITY;
// 10 - pentru prioritatea cea mai mare
public final static int MIN_PRIORITY;
// 1 - pentru prioritatea cea mai mica
public final static int NORM_PRIORITY;
// 5 - pentru prioritatea medie
```

O metodă utilă în aplicațiile cu fire de execuție este

```
public static native void yield()
```

care scoate procesul curent din execuție și îl pune în coada de așteptare.

În continuare prezentăm un exemplu care demonstrează cum se lucrează cu prioritățile firelor de execuție. Metoda `getName` (moștenită din clasa **Thread**) returnează numele procesului curent.

Exemplu

```
public class Fir2 {
    public static void main(String args[]) {
        FirDeExecutie fir1 = new FirDeExecutie("Fir 1");
        FirDeExecutie fir2 = new FirDeExecutie("Fir 2");
        FirDeExecutie fir3 = new FirDeExecutie("Fir 3");
        FirDeExecutie fir4 = new FirDeExecutie("Fir 4");
        fir1.setPriority(Thread.MIN_PRIORITY);
        fir2.setPriority(Thread.MAX_PRIORITY);
        fir3.setPriority(7);
        fir4.setPriority(3);
        fir1.start();
    }
}
```

```

        fir2.start();
        fir3.start();
        fir4.start();
        System.out.println("Revenirea la main");
    }
}

class FirDeExecutie extends Thread {
    public FirDeExecutie(String s) {
        super(s);
    }

    public void run() {
        String numeFir = getName();
        for (int i = 0; i<20; i++) {
            System.out.println(numeFir + " este la
pasul " + i);

            try {
                sleep(500);
            } catch (InterruptedException e) {
                System.err.println("Eroare");
            }
        }

        System.out.println(numeFir + " s-a
terminat");
    }
}

```

Observație

Implementările Java depind de platformă. Un program Java care folosește fire de execuție poate avea comportări diferite la execuții diferite, pentru aceleași date de intrare.

12.4 Crearea unui fir de execuție folosind interfața **Runnable**

Interfața **Runnable** este o modalitate utilă atunci când clasa de tip **Thread** care se dorește a fi implementată moștenește o altă clasă (precizăm faptul că Java nu permite moștenirea multiplă). Interfața **Runnable** descrie o singură metodă *run*.

Pentru crearea firelor de execuție folosind interfața **Runnable** se parcurg următoarele etape:

- se creează o clasă care implementează interfața **Runnable**
- se implementează metoda *run* din interfață
- se instanțiază un obiect al clasei create folosind *new*
- se creează un obiect din clasa **Thread** folosind un constructor care are ca parametru un obiect de tip **Runnable** (un obiect al clasei ce implementează interfața)
- se pornește **thread**-ul creat la pasul anterior prin apelul metodei **start()**.

Exemplu

```
public class Fir3 {  
    public static void main(String args[]) {  
        FirdeExecutie fir = new FirdeExecutie();  
        Thread thread = new Thread(fir);  
        thread.start();  
        System.out.println("Revenim la main");  
    }  
}
```

```

class A{
    public void afis(){
        System.out.println("Este un exemplu simplu");
    }
}

class FirdeExecutie extends A implements Runnable{
    public void run(){
        int i;
        for(i=0;i<20;i++)
            System.out.println("Pasul "+i);
        afis();
        System.out.println("run s-a incheiat");
    }
}

```

Observații

Un fir de execuție se poate afla la un moment dat în una dintre următoarele stări: *running* (rulează), *waiting* (adormire, blocare, suspendare), *ready* (gata de execuție, prezent în coada de așteptare), *dead* (terminat). Fiecare fir de execuție are o prioritate de execuție. În general *thread*-ul cu prioritatea cea mai mare este cel care va accesa primul resursele sistem.

O altă clasă aparte de *thread*-uri sunt cele *Daemon* care sunt *thread*-uri de serviciu (aflate în serviciul altor fire de execuție). Când se pornește mașina virtuală Java, există un singur fir de execuție care nu este de tip *Daemon* și care apelează metoda *main*. Mașina Virtuală Java rămâne pornită atât timp cât este activ un *thread* care să nu fie de tipul *Daemon*.

Probleme rezolvate

1. Se dă un vector cu n componente numere naturale. Determinați numărul de componente impare din vector.

Exemplu

Intrare 7 1 8 5 3 80 13 15	Ieșire 5 numere impare
----------------------------------	---------------------------

Soluție

Vom utiliza două fire de execuție, pentru a calcula numărul de numere impare din secvența cu indicii $0..n/2-1$, respectiv $n/2 .. n-1$.

Implementare cu clasa Thread

```
class FirDeExecutie1 extends Thread{  
    private int i, j; //indicii intre care se calculeaza  
                      //nr. de componente impare  
    private int[] x; //vect pt. care se realiz. calculele  
    private int cnt;  
    public FirDeExecutie1(int i, int j, int[] x){  
        this.i=i;  
        this.j=j;  
        this.x = x;  
    }  
    public boolean esteImpar(int x){  
        if(x%2==1)  
            return true;  
        return false;  
    }  
}
```

```

public int getContor(){
    return cnt;
}

public void run(){
    int k;
    for(k=i;k<=j;k++)
        if(estImpar(x[k]))
            cnt++;
}
}

public class NrImparVectorThread{
public static void main(Stringargs[]){
    int[] x ={1, 8, 5, 3, 80, 13, 15};
    FirDeExecutie1[] fir = new FirDeExecutie1[2];
    int cnt = 0, i, n = 7;
    fir[0] = new FirDeExecutie1(0, n/2-1, x);
    fir[0].start();
    try{
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    cnt += fir[0].getContor();
    fir[1] = new FirDeExecutie1(n/2, n-1, x);
    fir[1].start();
}
}

```

```

try{
    Thread.sleep(1000);
} catch (InterruptedException e) {
    e.printStackTrace();
}
cnt += fir[1].getContor();
System.out.println(cnt + " numere impare");
}
}

```

Implementare cu clasa Runnable.

```

class FirDeExecutie2 implements Runnable{
    private int i, j;
    private int[] x;
    private int cnt;
    public FirDeExecutie2(int i, int j, int[] x){
        this.i=i;
        this.j=j;
        this.x = x;
    }
    public boolean esteImpar(int x){
        if(x%2==1)
            return true;
        return false;
    }
}

```

```

        public int getContor(){
            return cnt;
        }
        public void run(){
            int k;
            for(k=i;k<=j;k++)
                if(estImpar(x[k]))
                    cnt++;
        }
    }

    public class NrImparVectorRunnable{
        public static void main(String[] args){
            int[] x = {1, 8, 5, 3, 80, 13, 15};
            FirDeExecutie2[] fir = new FirDeExecutie2[2];
            Int cnt = 0, i, n = 7;
            fir[0] = new FirDeExecutie2(0, n/2-1, x);
            Thread t = new Thread(fir[0]);
            t.start();
            try{
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            cnt += fir[0].getContor();
            fir[1] = new FirDeExecutie2(n/2, n-1, x);
            Thread tt = new Thread(fir[1]);
            tt.start();
        }
    }

```

```

try{
    Thread.sleep(1000);
} catch (InterruptedException e) {
    e.printStackTrace();
}
cnt += fir[1].getContor();
System.out.println(cnt + " numere impare");
}
}

```

2. Se dă un tablou bidimensional cu m linii și n coloane. Se cere să se determine numărul de componente impare din tablou.

Exemplu

Intrare	Ieșire
3 4	7 numere impare
1 20 5 256	
51 4 23 55	
1 2 3 4	

Soluție

Pentru fiecare linie i vom crea câte un fir de execuție, $i=0, \dots, m-1$. Valorile obținute de la fiecare fir de execuție vor fi adunate la o variabilă notată cu cnt.

Implementare cu clasa Thread

```

class FirDeExecutie3 extends Thread{
    private int linie;
    private int[] mat;
    private int cnt;
    public FirDeExecutie3(int linie, int[] mat){
        this.linie = linie;
    }
}

```

```

        this.mat = mat;
    }
    public boolean esteImpar(int x){
        if(x%2==1)
            return true;
        return false;
    }
    public int getContor(){
        return cnt;
    }
    public void run(){
        int i;
        for(i=0;i<linie;i++)
            if(esteImpar(mat[i]))
                cnt++;
    }
}

public class NrImparTabThread{
    public static void main(String[] args){
        int[][] a ={
            {1, 20, 5, 256},
            {51, 4, 23, 55},
            {1, 2, 3, 4}
        };
        FirDeExecutie3[] fir = new FirDeExecutie3[a.length];
        int cnt = 0, i;

```

```

for(i=0;i<a.length;i++){
    fir[i] = new FirDeExecutie3(a[i].length, a[i]);
    fir[i].start();
    try{
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    cnt += fir[i].getContor();
}
System.out.println(cnt + " numere impare");
}
}

```

Implementare cu interfața Runnable.

```

class FirDeExecutie4 implements Runnable{
    private int linie;
    private int[] mat;
    private int cnt;
    public FirDeExecutie4(int linie, int[] mat){
        this.linie = linie;
        this.mat = mat;
    }
    public boolean esteImpar(int x){
        if(x%2==1)
            return true;
        return false;
    }
}

```

```

public int getContor(){
    return cnt;
}
public void run(){
    int i;
    for(i=0;i<linie;i++)
        if(estImpar(mat[i]))
            cnt++;
}
}
public class NrImparTabRunnable{
public static void main(Stringargs[]){
int[][] a ={
                {1, 20, 5, 256},
                {51, 4, 23, 55},
                {1, 2, 3, 4}
            };
FirDeExecutie4[] fir = new FirDeExecutie4[a.length];
int cnt = 0, i;
Thread[] t = new Thread[a.length];
for(i=0;i<a.length;i++){
    fir[i] = new FirDeExecutie4(a[i].length, a[i]);
    t[i] = new Thread(fir[i]);
    t[i].start();
    try{
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    cnt += fir[i].getContor();
}
System.out.println(cnt + " numere impare");
}
}

```


Probleme propuse spre rezolvare folosind fire de execuție prin extinderea clasei Thread

1. Se dă un tablou bidimensional cu m linii și n coloane cu componente întregi. Determinați numărul de componente prime.

Exemplu

Intrare	Ieșire
3 4	5
1 20 5 256	
50 4 23 55	
7 2 3 4	

2. Se dă un tablou bidimensional cu m linii și n coloane cu component întregi. Ordonăți crescător liniile tabloului. Apoi afișați tabloul.

Exemplu

Intrare	Ieșire
3 4	1 5 20 256
1 20 5 256	4 23 51 55
51 4 23 55	1 2 3 4
1 4 3 2	

3. Se dă un vector cu n component întregi. Înlocuiți fiecare componentă cu cel mai mare pătrat perfect mai mic sau egal decât componenta. Apoi afișați vectorul.

Exemplu

Intrare	Ieșire
4	64 16 81 121
80 23 100 125	

Probleme propuse spre rezolvare folosind fire de execuție prin implementarea interfeței Runnable

1. Se dă un tablou bidimensional cu m linii și n coloane cu componente întregi. Determinați numărul de componente cu prima cifră pară.

Exemplu

Intrare	Ieșire
3 4	5
1 20 5 256	
50 4 23 55	
1 2 3 4	

2. Se dă un tablou bidimensional cu m linii și n coloane cu component întregi. Pentru fiecare linie scrieți unul din mesaje: DA sau NU. DA, dacă linia are toate componentele pare, respectiv NU contrar.

Exemplu

Intrare	Ieșire
3 4	DA
10 2 50 250	NU
51 4 23 55	DA
16 2 34 20	

3. Se dă un vector cu n componente întregi. Înlocuiți fiecare componentă cu produsul cifrelor nenule. Apoi afișați vectorul.

Exemplu

Intrare	Ieșire
4	8 6 1 15
80 23 101 315	

13. Applet-uri cu fire de execuție

13.1 Noțiunea de applet

Un *applet* reprezintă o suprafață de afișare (container) ce poate fi inclusă într-o pagină web și gestionată printr-un program Java. Un astfel de program se mai numește *miniaplicație* sau prin abuz de limbaj, *applet*.

Codul unui applet poate fi format din una sau mai multe clase. Una dintre acestea este principală și extinde clasa *Applet*, fiind clasa ce trebuie specificată în documentul HTML ce descrie pagina web în care dorim să includem *applet-ul*.

Diferența fundamentală dintre un *applet* și o aplicație constă în faptul că, un applet nu poate fi rulat independent, ci va fi rulat de browser-ul în care este încărcată pagina Web ce conține *applet-ul* respectiv. O aplicație independentă este executată prin apelul interpretorului java, având ca parametru numele clasei principale a aplicației, clasa principală fiind cea care conține metoda *main*. Ciclul de viață al unui *applet* este complet diferit, fiind dictat de evenimentele generate de către browser la vizualizarea documentului HTML ce conține *applet-ul*.

Pachetul care oferă suport pentru crearea de *applet-uri* este *java.applet*.

Exemplu

Codul applet-ului salvat în fișierul *AppletSimplu.java* este următorul:

```
import java.applet.Applet;
import java.awt.*;

public class AppletSimplu extends Applet {
    public void paint(Graphics g) {
```

```

        g.setFont(new Font("Arial", Font.BOLD, 16));
        g.drawString("Salut!", 0, 30);
    }
}

```

Codul HTML ce lansează în executare applet-ul salvat în fișierul FolosireApplet.html este următorul:

```

<HTML>

<HEAD>

<TITLE> Un applet simplu </TITLE>

</HEAD>

<APPLET CODE="AppletSimplu.class" WIDTH=100
HEIGHT=50></APPLET>

</HTML>

```

Lansarea în executare a unui applet presupune folosirea aplicației appletviewer.

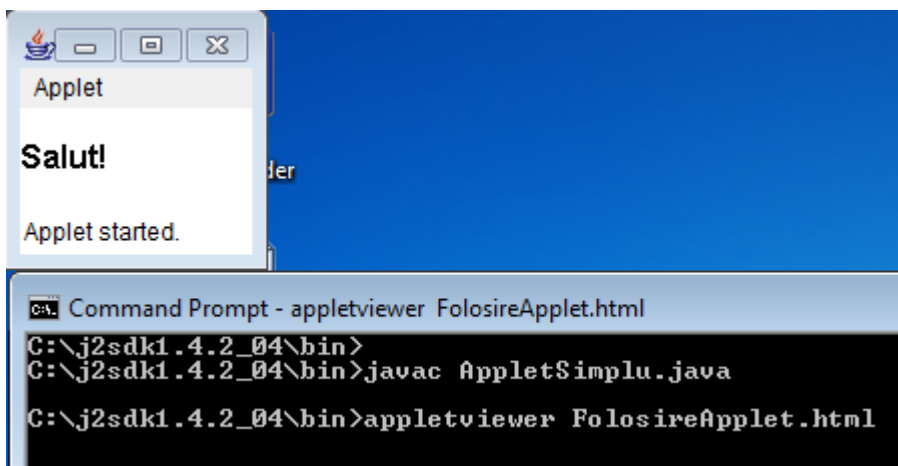
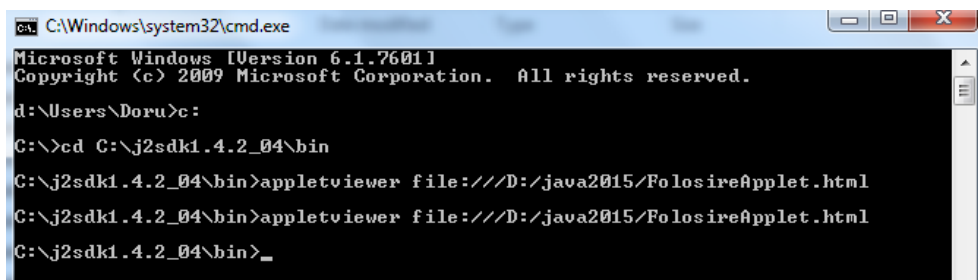


Figura 13.1 Lansarea în executare a unui applet din același folder cu cel în care se găsește compilatorul



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

d:\Users\Doru>c:
C:\>cd C:\j2sdk1.4.2_04\bin
C:\j2sdk1.4.2_04\bin>appletviewer file:///D:/java2015/FolosireaApplet.html
C:\j2sdk1.4.2_04\bin>appletviewer file:///D:/java2015/FolosireaApplet.html
C:\j2sdk1.4.2_04\bin>_
```

Figura 13.2 Lansarea în execuție a unui applet din alt folder decât cel în care se găsește compilatorul

Structura generala a unui applet

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class StructuraApplet extends Applet {
    public void init() {
    }
    public void start() {
    }
    public void stop() {
    }
    public void destroy() {
    }
    public void paint(Graphics g) {
    }
}
```

14.2 Fire de execuție în applet-uri

Fiecare applet aflat pe o pagina Web se execută într-un fir de execuție propriu. Acesta este creat de către browser și este responsabil cu desenarea appletului (apelul metodelor *update* și *paint*) precum și cu transmiterea mesajelor generate de către componentele appletului. În cazul în care dorim să realizăm și alte operații consumatoare de timp este recomandat să le realizăm într-un alt fir de execuție, pentru a nu bloca interacțiunea utilizatorului cu appletul sau redesenarea acestuia.

Structura unui applet care lansează un fir de execuție este următoarea:

```
import java.applet.Applet;

class AppletThread1 extends Applet implements Runnable
{
    ThreadappletThread = null;

    public void init() {
        if (appletThread == null) {
            appletThread = new Thread(this);
            appletThread.start();
        }
    }

    public void run() {
        //codul firului de executie
    }
}
```

Exemplu

Aplicație cu fire de execuție care afișează numerele naturale mai mici sau egale cu 99 (*AppletThread1.java*).

```

import java.applet.Applet;

import java.awt.*;

public class AppletThread1 extends Applet implements
Runnable {

    Thread appletThread = null;

    String s = "";

    int ok = 0;

    public void paint(Graphics g) {

        g.setFont(new Font("Arial", Font.BOLD, 16));

        if (appletThread == null) {

            appletThread = new Thread(this);

            appletThread.start();

        }

        while(ok==0);

        g.drawString(s, 0, 30);

    }


    public void run() {

        int i;

        for(i=1;i<=100;i++)

            s=s+i+" ";

        ok=1;

    }

}

```

Codul HTML pentru lansarea în executare a applet-ului (AppletThread1.html) este următorul:

```

<HTML>

<HEAD>

<TITLE> Un applet simplu </TITLE>

</HEAD>

<APPLET CODE="AppletThread1.class" WIDTH=100
HEIGHT=50></APPLET>

</HTML>

```

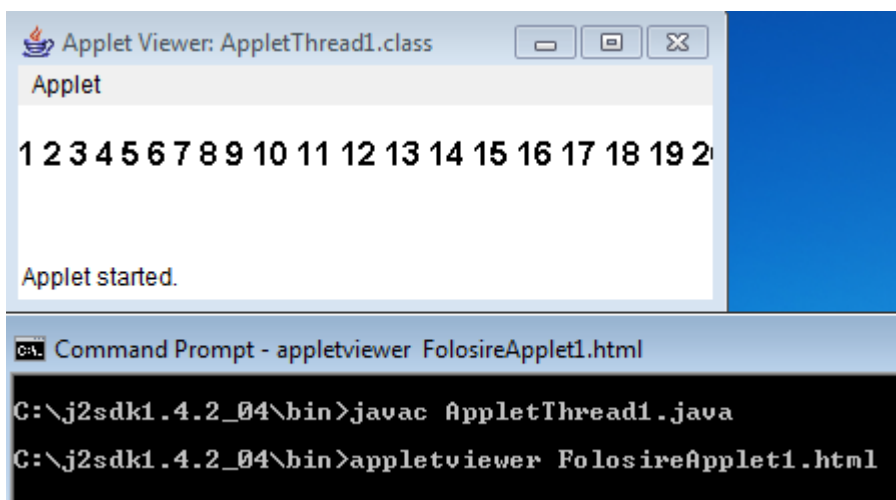


Figura 13.3 Compilarea și lansarea în execuție a unui applet cu fire de execuție

Observații

1. Un applet nu poate să:

- citească sau scrie fișiere pe calculatorul pe care a fost încărcat (*client*);
- deschidă conexiuni cu alte mașini în afară de cea de pe care provine (*host*);
- pornească programe pe mașina client
- citească diverse proprietăți ale sistemului de operare al clientului

2. Ferestrele folosite de un applet, altele decât cea a browserului, vor arăta altfel decât într-o aplicație obișnuită.

Imagini și desene în applet-uri

Afișarea imaginilor într-un applet se face fie prin intermediul unei componente ce permite acest lucru, cum ar fi o suprafață de desenare de tip *Canvas*, fie direct în metoda *paint* a applet-ului, folosind metoda *drawImage* a clasei *Graphics*. În ambele cazuri, încărcarea imaginii în memorie se va face cu ajutorul metodei *getImage* din clasa *Applet*. Aceasta poate primi ca argument fie adresa URL absolută a fișierului ce conține imaginea, fie calea sa relativă la o anumită adresă URL, cum ar fi cea a folderului în care se găsește documentul HTML ce conține appletul (*getDocumentBase*) sau a folderului în care se găsește clasa appletului (*getCodeBase*).

```
import java.applet.Applet;

import java.awt.*;

public class AppletImagine extends Applet {

    Image img = null;

    public void init() {

        img = getImage(getCodeBase(),
            "image.gif");
    }

    public void paint(Graphics g) {

        g.drawImage(img, 0, 0, this);

    }

}
```

Fișierul *.class* trebuie să se afle în același folder cu fișierul HTML, altfel nu este suficient să se scrie doar numele său la atributul *CODE*. Clasa principală a unui *applet* trebuie neapărat să extindă clasa predefinită *Applet*. Metoda *paint* primește ca parametru o referință a unui obiect de tip *Graphics* (clasă predefinită în *java.awt*), creat automat în momentul lansării în execuție a applet-ului.

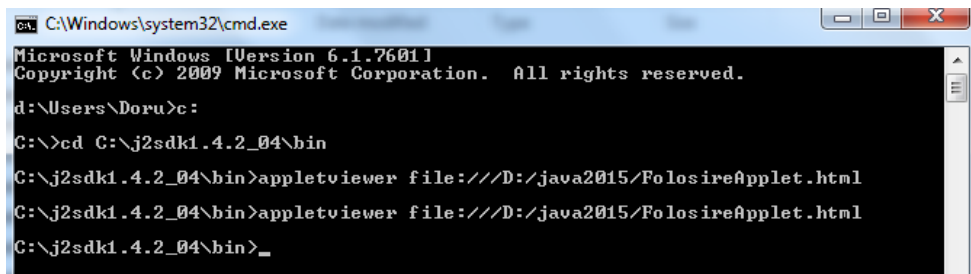
Un obiect *Graphics* poate realiza operații de desenare într-o porțiune a ecranului. După ce se operează modificări într-un applet, el trebuie recompilat, rezultând un fișier.class modificat. Pentru ca în pagina web, care include applet-ul respectiv, să fie vizibile modificările, este necesară acționarea butonului *Reload* al browser-ului în timp ce se ține apăsată tasta *Shift*.

Folosind clasele *Font* și *Color* din pachetul *java.awt.* se poate face applet-ul mai atractiv. Cu ajutorul obiectelor *Color* se pot stabili culori pentru umplerea diverselor forme geometrice sau pentru text. O culoare este specificată cu ajutorul a trei numere întregi din intervalul [0, 255] care indică nivelul de roșu, verde și albastru (RGB) prin combinația cărora va rezulta culoarea dorită. Spre exemplu, pentru a scrie textul cu violet, applet-ulse va scrie astfel:

```
import java.awt.*;
import java.applet.*;
public class PrimulApplet extends Applet{
    public void paint (Graphics g){
        Color c = new Color(180,10,120);
        g.setColor(c);
        g.drawString("Hello World", 20, 20);
    }
}
```

Există un set de obiecte de tip *Color* definite ca date membre statice ai clasei *Color*:

Color.black (negru), *Color.blue* (albastru), *Color.darkGray* (gri închis), *Color.gray* (gri), *Color.green* (verde), *Color.red* (roșu), *Color.white* (alb), *Color.yellow* (galben), etc.



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

d:\Users\Doru>c:
C:\>cd C:\j2sdk1.4.2_04\bin
C:\j2sdk1.4.2_04\bin>appletviewer file:///D:/java2015/FolosireApplet.html
C:\j2sdk1.4.2_04\bin>appletviewer file:///D:/java2015/FolosireApplet.html
C:\j2sdk1.4.2_04\bin>_
```

Figura 13.4 Lansare în executare a unui applet din alt folder decât cel în care se găsește compilatorul

Pentru specificarea unui anumit font pentru caracterele textului se utilizează clasa `Font` cu precizarea a trei parametri:

- tipul fontului (de exemplu `"TimesRoman"`);
- stilul, sub forma unor constante predefinite: `Font.PLAIN` (caractere normale), `Font.BOLD` (caractere îngroșate) sau `Font.ITALIC` (caractere înclinate).

Pentru combinarea stilurilor se utilizează operatorul sau `"|"`;

- dimensiunea caracterelor, exprimată ca număr de puncte: 10, 12, 14, 18 etc.

Pentru a folosi fonturi *Times New Roman*, bold și cu dimensiunea de 18 puncte, exemplul se modifică astfel:

```
import java.awt.*;
import java.applet.*;
public class PrimulApplet extends Applet{
    public void paint (Graphics g){
        Color c = new Color(180,10,120);
        g.setColor(c);
        Font f = new Font("TimesRoman", Font.BOLD, 18);
        g.drawString("Hello World", 20, 20);
    }
}
```

Dacă într-un *applet* se execută operațiile `setColor` sau `setFont`, valorile stabilite de către acestea pentru culoare și font vor rămâne active până la sfârșitul programului sau până la o nouă modificare. Pentru a obține informații despre culoarea/fontul curente se pot folosi următoarele metode:

— `getColor` returnează un obiect de tip `Color` reprezentând culoarea curentă. Pentru a afla componentele RGB ale culorii se vor aplica metodele `getRed`, `getGreen` și `getBlue`.

— `getFont` care returnează referința unui obiect `Font`, reprezentând fontul curent.

Pe lângă metode pentru afișarea de texte, clasa `Graphics` include și metode de desenare a unor forme geometrice. Cele mai utilizate metode de desenare sunt:

— `drawRect(int x, int y, int width, int height)` desenează conturul unui dreptunghi cu colțul din stânga sus în punctul de coordonate (x, y) , lungimea egală cu *width* și înălțimea egală cu *height*. Culoarea utilizată pentru contur este culoarea curentă;

— `drawLine(int x1, int y1, int x2, int y2)` desenează un segment de dreaptă ale cărei capete au coordonatele $(x1, y1)$, respectiv, $(x2, y2)$. Culoarea utilizată pentru linie este culoarea curentă;

— `drawOval(int x, int y, int width, int height)` desenează conturul unei elipse având cele două diametre principale paralele cu laturile *applet*-ului. (x, y) reprezintă coordonatele colțului din stânga sus ale dreptunghiului imaginar care circumscrie elipsa, iar *width* și *height* reprezintă lărgimea, respectiv înălțimea elipsei. Culoarea utilizată pentru contur este culoarea curentă;

— `fillRect(int x, int y, int width, int height)` colorează interiorul unui dreptunghi cu culoarea curentă. Semnificația parametrilor este aceeași ca și la `drawRect`;

— `fillOval(int x, int y, int width, int height)` colorează interiorul unei elipse cu culoarea curentă. Semnificația parametrilor este aceeași ca la metoda `drawOval`.

Exemplu

Afișarea unei imagini și a unui text încercuit:

```
import java.awt.* ;  
import java.applet.* ;
```

```

public class Applet2 extends Applet {
    Image img;
    public void init() {
        img = getImage(getCodeBase(), "Penguins.gif");
    }
    public void paint (Graphics g) {
        //g=suprafata pe care se poate desena
        g.drawImage(img, 0, 0, this);
        g.drawOval(300,0,275,100);
        g.drawString("O imagine placuta!", 350, 55);
    }
}

```

Apoi:

```

<HTML>
<HEAD>
<TITLE> Un applet simplu </TITLE>
</HEAD>
<APPLET CODE="Applet2.class" WIDTH=400
HEIGHT=350></APPLET>
</HTML>

```

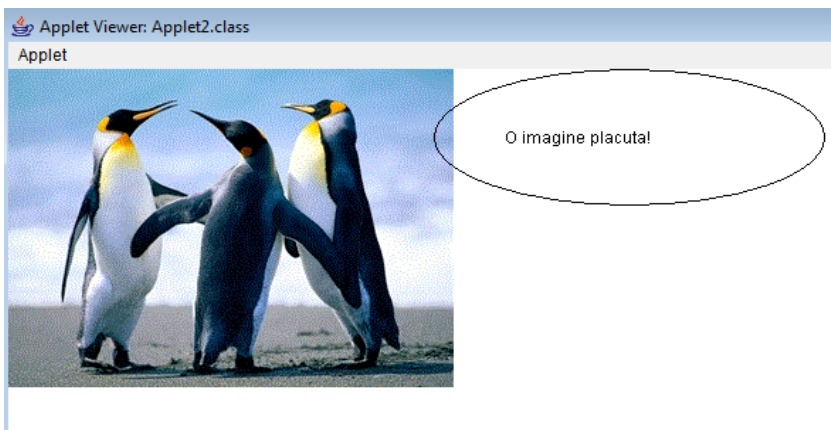


Figura 13.5 Rezultatul lansării în executare a applet-ului

Probleme propuse

1. Creați un applet, care să afișeze numele și prenumele vostru împreună cu adresa de email.
2. Creați un applet, care să afișeze o imagine și sub ea ce reprezintă. Exemplu: Imagine cu o personalitate științifică și numele ei.
3. Folosind fire de execuție creați un applet, care să afișeze pătratele numerelor naturale mai mici sau egale cu 100.
4. Folosind fire de execuție creați un applet, care să afișeze numerele *superprime* < 10000 (numărul natural k este *superprim*, dacă k și suma cifrelor lui k sunt numere prime).
5. Folosind fire de execuție creați un applet, care să afișeze fiecare literă mare din alfabetul englez de atâtea ori cât este poziția sa în alfabet.
6. Folosind fire de execuție creați un applet care să afișeze toate anagramele cuvântului *mere*.
7. Creați un applet care să conțină un pătrat și o minge de culoare galbenă, ce se deplasează din colțul din dreapta sus în colțul din stânga jos.
8. Creați un applet care să conțină un pătrat și o minge de culoare roșie, ce se deplaseze din colțul din dreapta jos în colțul din stânga sus.
9. Creați un applet care să conțină un dreptunghi și o elipsă de culoare roșie, ce se deplasează pe conturul dreptunghiului.
10. Creați un applet care să afișeze pe mai multe linii paralele mingii umplute cu culori diferite, ce se deplasează de la stânga la dreapta.
11. Creați un applet care să afișeze pe mai multe coloane paralele mingii umplute cu culori diferite, ce se deplasează de sus în jos.

Bibliografie

1. Tudor Sorin, *Bazele programării în Java*, Editura L&S Infomat, 2012
2. Vlad Tudor (Huțanu), Carmen Popescu, *Manual de INFORMATICĂ pentru clasa a XII-a, profilul real-intensiv*, Editura L&S Infomat, 2016
3. Carmen Popescu, *INFORMATICĂ, Culegere de probleme pentru clasele IX-XI*, Editura L&S Infomat, 2017
4. Doru Anastasiu Popescu, *Programare Paralelă – note de curs*, UPIT, 2019
5. Doru Anastasiu Popescu, *An Implementation of the Greedy Algorithm for Multicore Systems*, University of Pitești Scientific Bulletin, Series: Electronics and Computers Science, Vol 14, Issue 2, pp. 1-4, 2014
6. Herbert Schildt, *Java™: The Complete Reference*, Seventh Edition, McGraw-Hill 2007
7. Horia Georgescu, *Introducere în universul Java*, Editura Tehnică, 2002

<https://ebooks.infobits.ro>